SUP'COM

Higher School of Communication of Tunis

Compte Rendu

Intrusion Detection

Lab 2 – Network Intrusion Detection using Suricata

Student: Mohamed Mensi
Program: M2 Cybersecurity Engineering – Operational Cybersecurity
Professor: Slim Rekhis

Academic year: 2026/2027
Date: October 29, 2026

Contents

I Lab Objective **3**

II Installing Suricata and Dependencies **3**

III Suricata Configuration and Network Setup **5**

IV Testing Suricata's Built-In Rules **9**
IV.1 Detection of Root-Level Compromise . . . 9
IV.2 Detection of Malware User-Agents (BlackSun) . . . 11
IV.3 Detection of Malicious IP Sources (Spamhaus DROP) . . . 14

V Adding Our Own Rules to Suricata **17**
V.1 Detecting Echo Replies . . . 17
V.2 Detecting ICMP Flood / Denial of Service . . . 18
V.3 Detecting PDF File Transfer . . . 21
V.4 Detecting Embedded JavaScript in PDF Files . . . 23
V.5 Detecting EXE File Transfer . . . 26
V.6 Detecting Clear-Text Card Numbers (DLP) . . . 28
V.7 Analyzing a PCAP File and Detecting Phishing . . . 32

VI Summary of Custom Rules and Alerts **34**

VII Conclusion **34**

List of Figures

1 Output of `suricata -V` . . . 4
2 Output of `suricata -build-info` . . . 4
3 Verification of Suricata files and directories . . . 5
4 Active interface identified (`ip -br addr` and `ip route`) . . . 6
5 `af-packet` section of `suricata.yaml` . . . 6
6 `vars` section: `HOME_NET`, `EXTERNAL_NET`, server groups . . . 7
7 `default-rule-path` and `rule-files` . . . 7
8 Output of `suricata-update` . . . 8
9 Number of rules in `suricata.rules` . . . 8
10 Suricata service stopped and disabled . . . 9
11 Successful configuration test and start (`-T -v`) . . . 9
12 Rule SID 2100498 found in `suricata.rules` . . . 10
13 `curl` to the NIDS test page . . . 10
14 Alert generated for SID 2100498 in `fast.log` . . . 11
15 `curl` with User-Agent `BlackSun` . . . 13
16 Alert for SID 2008983 . . . 13
17 `fast.log` after both commands . . . 14
18 Spamhaus DROP rule in `suricata.rules` . . . 15
19 SYN flood launched with `hping3` . . . 16
20 Spamhaus DROP alerts in `fast.log` . . . 16
21 `custom.rules` added under `rule-files` . . . 17
22 Ping to 8.8.8.8 . . . 18
23 Alert for SID 1000001 . . . 18
24 ICMP flood generation . . . 19

25 Alert for SID 1000002 . . . 20
26 PDF opened in `bless`: signature `25 50 44 46 2D` . . . 21
27 Packets containing the `%PDF-` header . . . 22
28 Alert for SID 1000003 . . . 22
29 HTTP server running on the Windows machine . . . 25
30 Download on the Suricata host . . . 25
31 Alert for SID 1000007 . . . 26
32 PE header of the WildFire sample (optional evidence) . . . 27
33 Alert generated by the EXE rule . . . 28
34 SMTP packet 15 showing the card numbers . . . 29
35 Reconstructed email (Follow TCP Stream) . . . 30
36 Alert for SID 1000009 . . . 30
37 DNS query in packet 29 . . . 32
38 Alert for SID 1000010 . . . 33

I. Lab Objective

This lab covered the installation and configuration of Suricata on Ubuntu 22.04, including the integration of the Emerging Threats ruleset. We developed custom detection rules and validated generated alerts using Wireshark and tcpreplay. The exercises demonstrated how signature-based detection identifies known threats in network traffic. However, its effectiveness can be limited by encryption, evasion techniques, and false positives or negatives.

Note on the platform. The lab is written for Ubuntu 22.04. This report was carried out on **Kali Linux**. Commands are given for both systems where they differ: **[Ubuntu 22.04]** marks the lab's original command and **[Kali Linux]** its Kali equivalent. Commands without a tag are identical on both. The interface name `enp0s3` used in the lab must be replaced by the actual interface of the Kali VM (`eth0`).

Lab environment

Item	Value
Hypervisor	VirtualBox
Suricata VM OS	Kali Linux 2025.4 (lab written for Ubuntu 22.04)
Suricata installed from	apt
Monitored interface	eth0
VM IP address / subnet	10.0.2.0/24
Suricata version	8.0.7
Windows machine IP (Section V.4)	192.168.1.240

Table 1: Lab environment summary

II. Installing Suricata and Dependencies

Step 1–4. Installation of utilities and Suricata.

[Ubuntu 22.04] (as in the lab: OISF stable PPA)

```
$ sudo apt update
$ sudo apt install -y software-properties-common curl wget unzip jq ethtool wireshark tcpreplay
    hping3 bless
$ sudo add-apt-repository -y ppa:oisf/suricata-stable
$ sudo apt update
$ sudo apt install suricata
```

[Kali Linux] (the PPA is Ubuntu-specific and is not used; Suricata comes from the Kali repositories)

```
$ sudo apt update
$ sudo apt install -y curl wget unzip jq ethtool wireshark tcpreplay hping3 xxd
$ sudo apt install -y suricata
$ apt policy suricata # installed version and origin
```

Step 5. Installed version and build information.

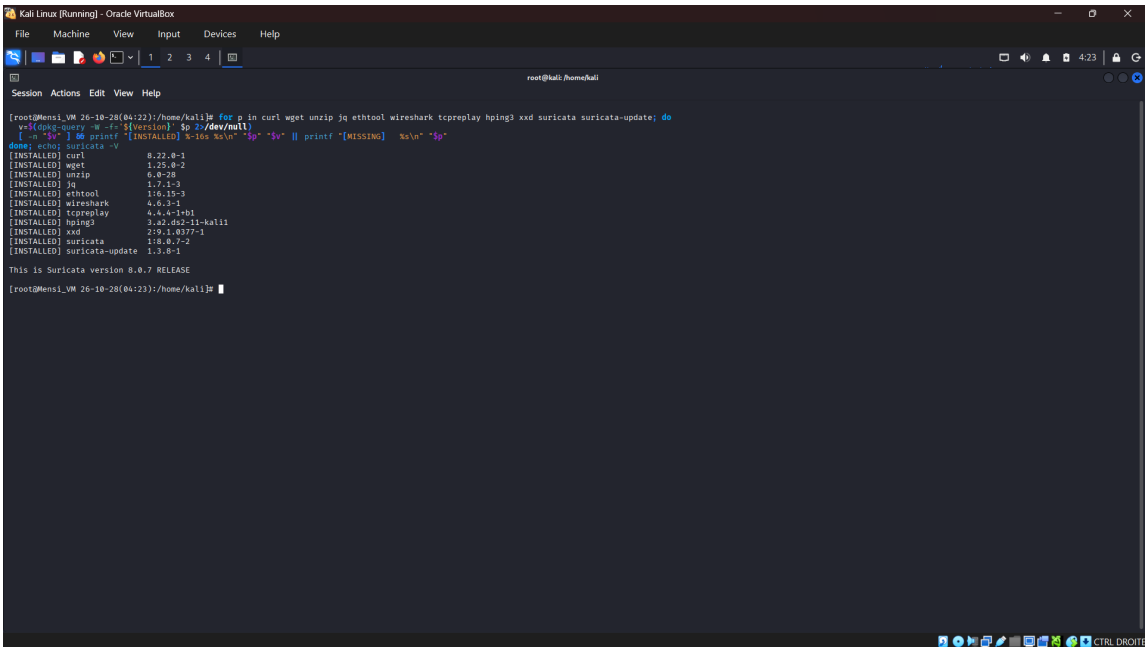


Figure 1: Output of `suricata -V`

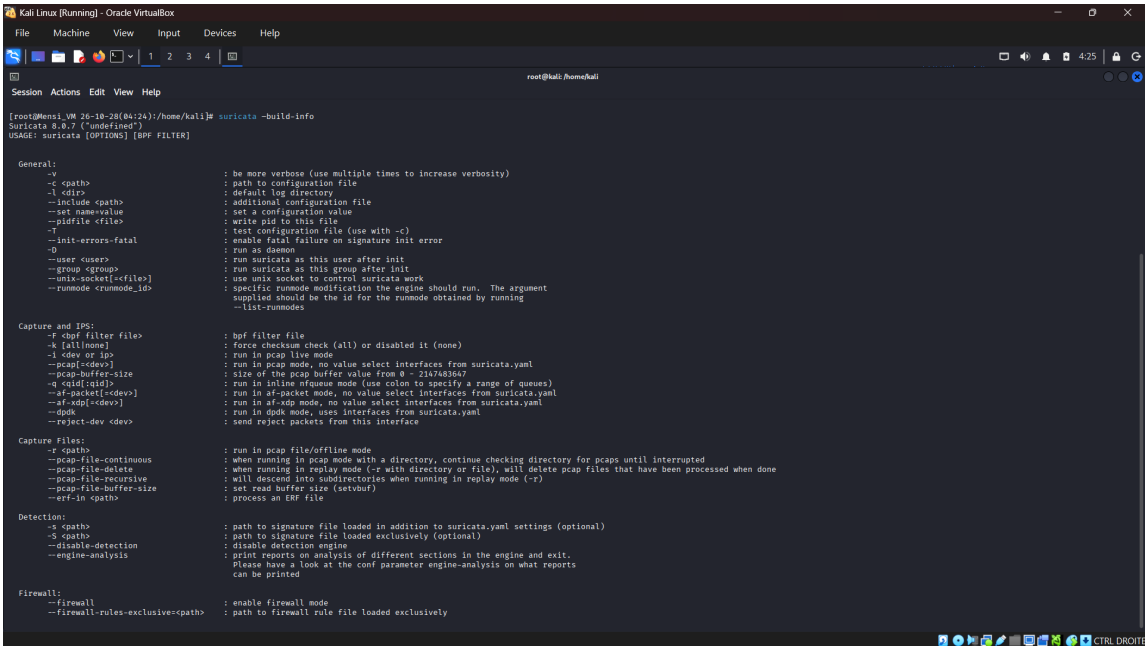


Figure 2: Output of `suricata -build-info`

Step 6. Verification of files and directories.

Path	Role
`/etc/suricata/reference.config`	Defines the reference types that rules can cite with the `reference` keyword (e.g. `url`, `cve`, `bugtraq`) and the URL prefix used to build the link for each type.
`/etc/suricata/classification.config`	Defines the classification types (`classtype`) used by rules, such as `trojan-activity` or `attempted-recon`, with a description and a default priority for each. This is what gives alerts their "Classification" and "Priority" fields.
`/var/log/suricata/`	Directory where Suricata writes its logs and output files: `fast.log` (one-line alerts), `eve.json` (detailed JSON events), `stats.log` (performance counters) and `suricata.log`.
`/etc/suricata/suricata.yaml`	Main configuration file. It sets the network variables (`HOME_NET`, `EXTERNAL_NET`), the capture method and interface (`af-packet`), the rule path and rule files, and the logging outputs.

Table 2: Suricata files and directories

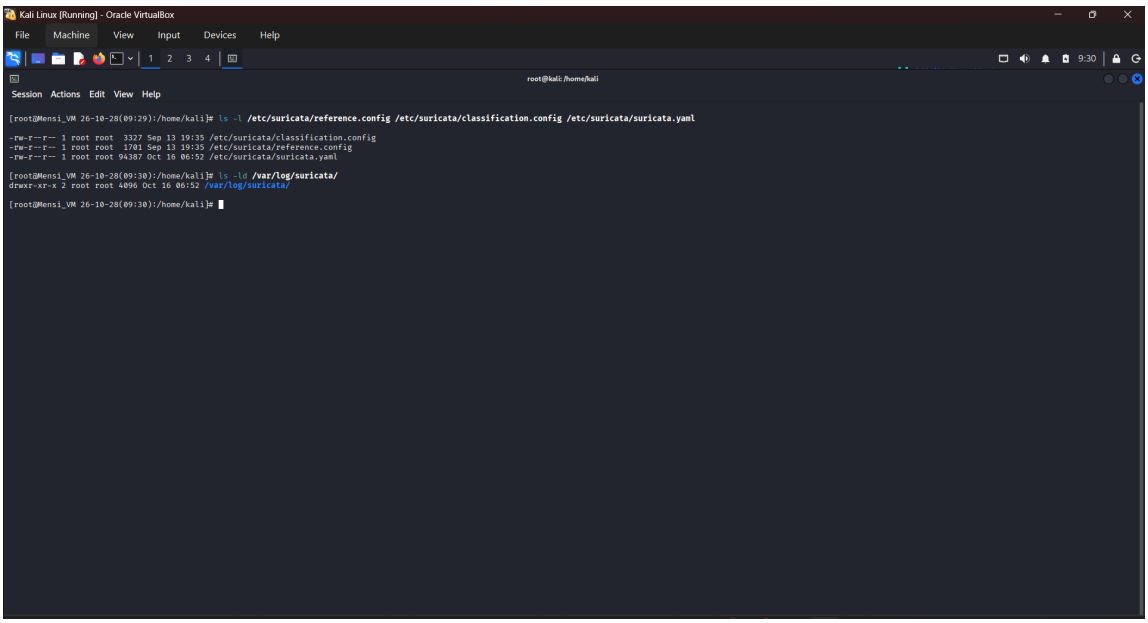


Figure 3: Verification of Suricata files and directories

III. Suricata Configuration and Network Setup

Step 7. Backup of the configuration file and identification of the active interface.

```
$ sudo cp /etc/suricata/suricata.yaml /etc/suricata/suricata.yaml.backup
$ ip -br addr
$ ip route
```

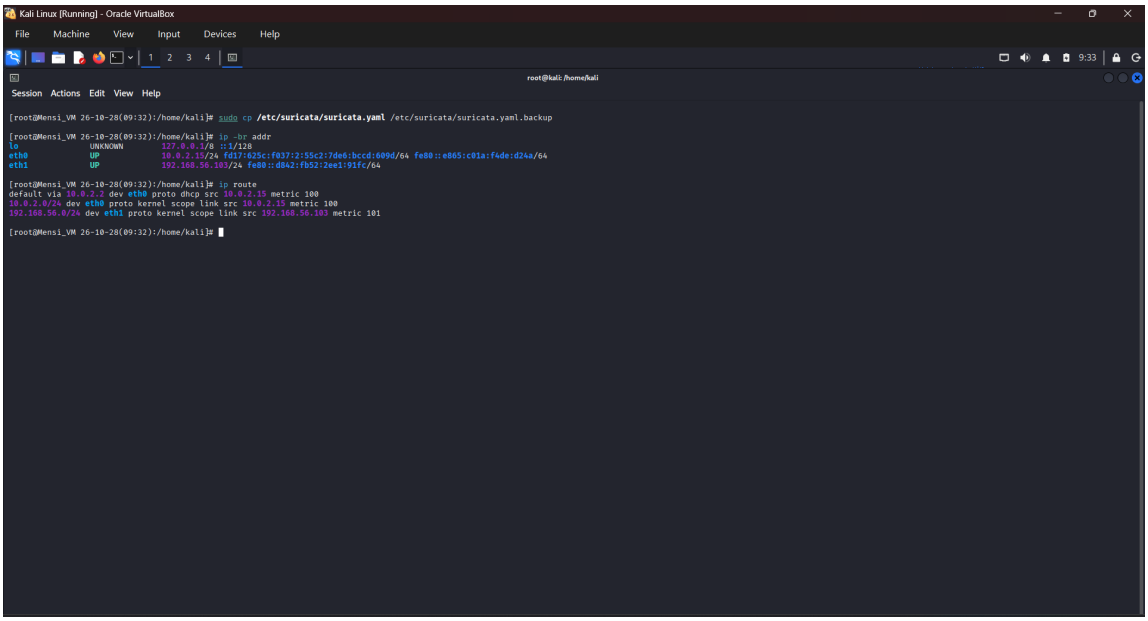


Figure 4: Active interface identified (`ip -br addr` and `ip route`)

Interface on my Kali VM:eth0. **In all later commands and in the `af-packet` section, `enp0s3` is replaced by this name.**

Step 8. Configuration of the `af-packet` section.

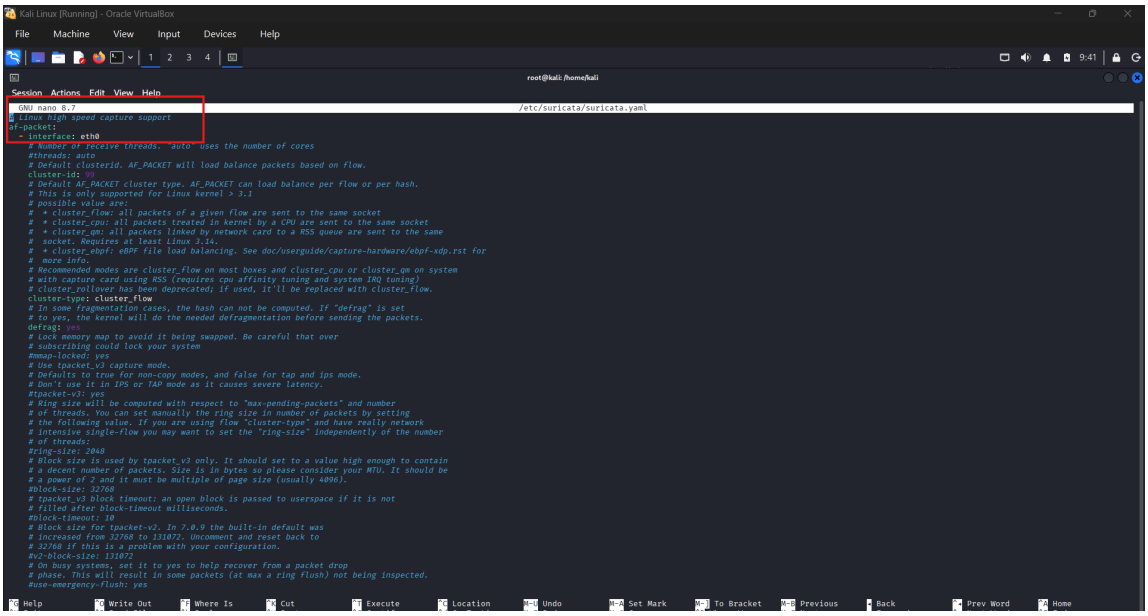


Figure 5: `af-packet` section of `suricata.yaml`

Step 9–10. `HOME_NET`, `EXTERNAL_NET` and server variables. my eth0 interface belong to the 10.0.2.0/24 network and the suricata config the HOME-NET variable already include my network with the 10.0.0.0/8 network and EXTERNAL-NET is set to whatever not in my HOME-NET

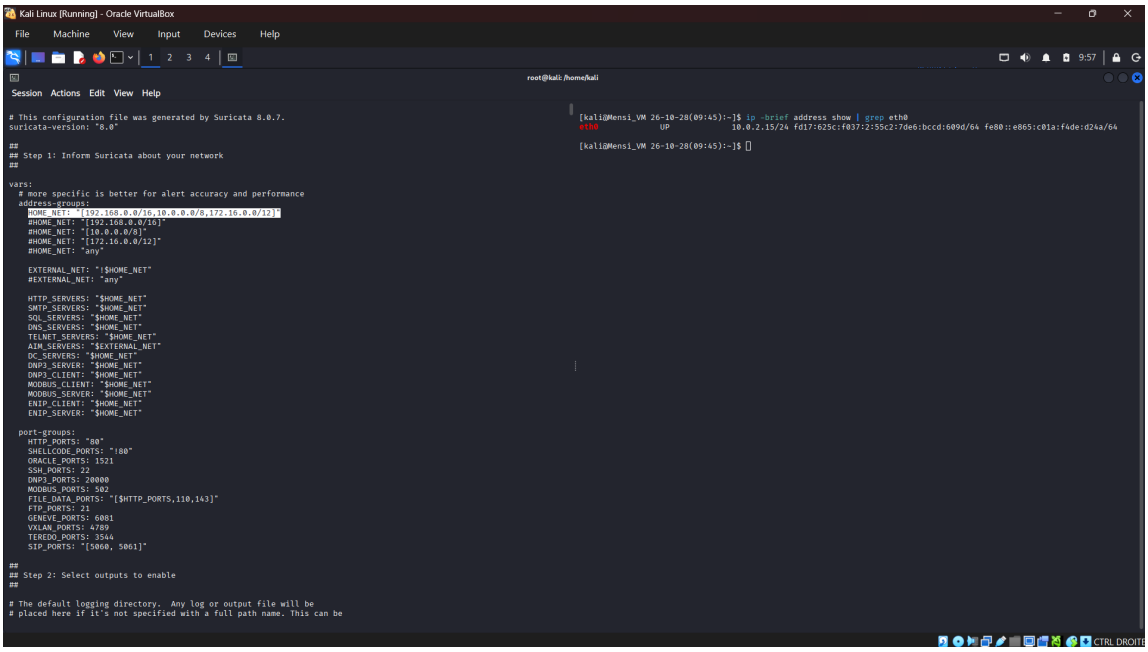


Figure 6: `vars` section: `HOME_NET`, `EXTERNAL_NET`, server groups

Step 11. Rule path and rule files.

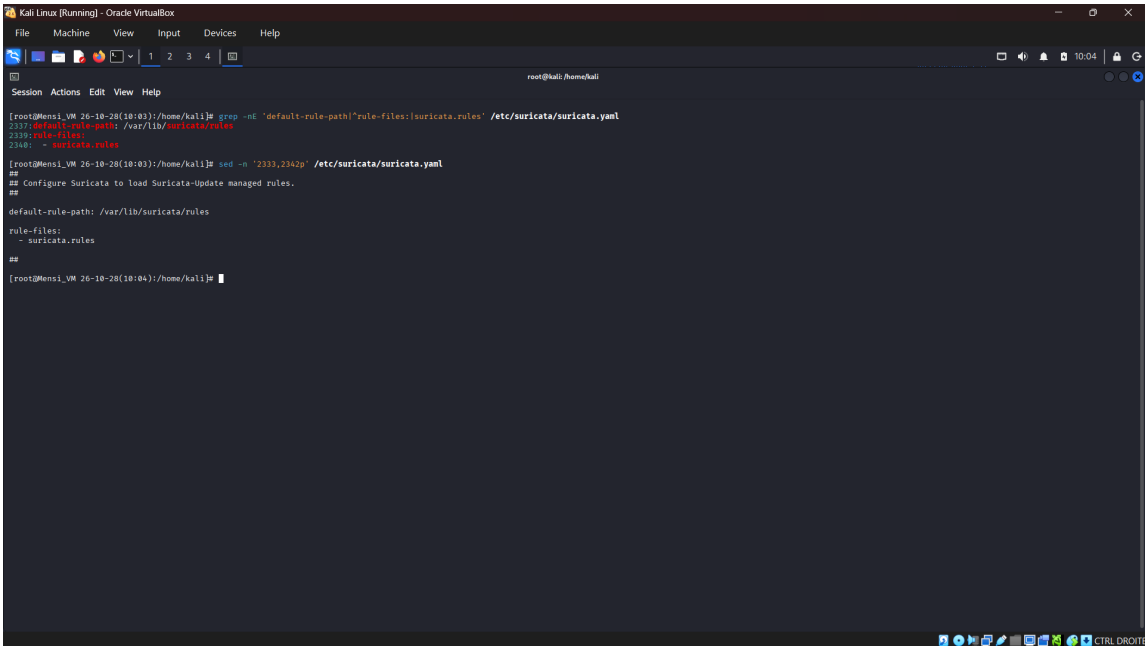


Figure 7: `default-rule-path` and `rule-files`

Step 12–14. Emerging Threats ruleset via `suricata-update`.

```
$ sudo suricata-update
# wc -l /var/lib/suricata/rules/suricata.rules
```

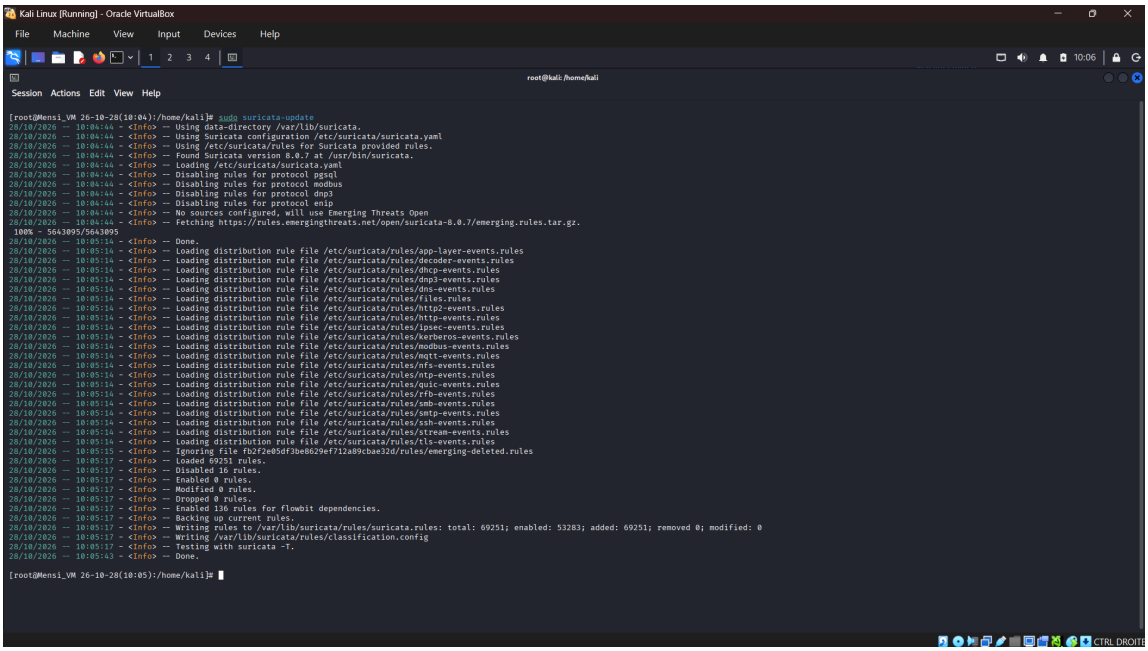


Figure 8: Output of `suricata-update`

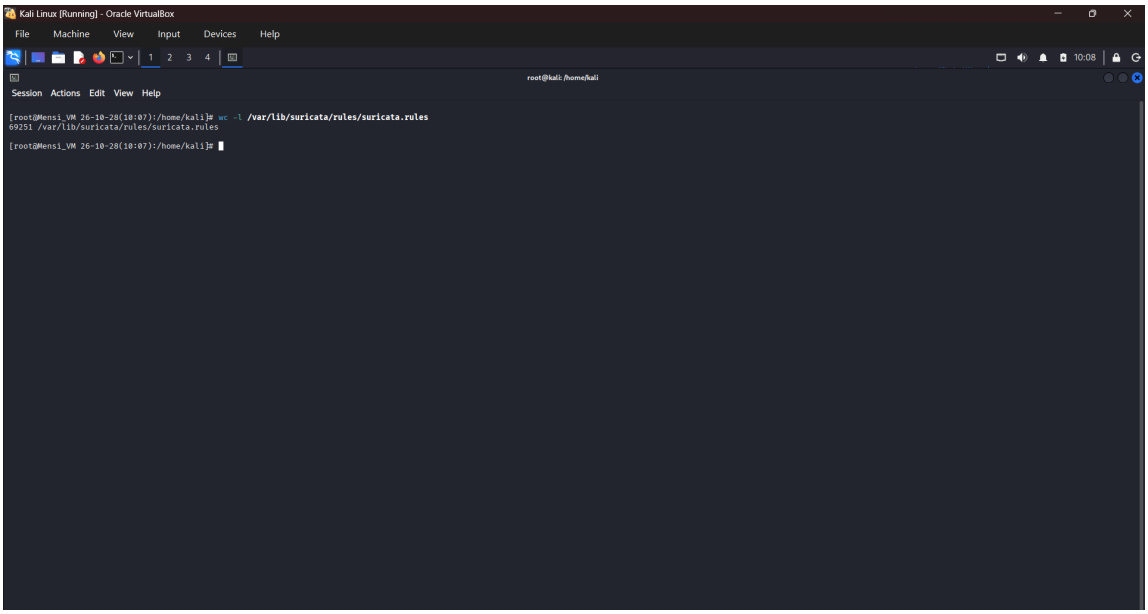


Figure 9: Number of rules in `suricata.rules`

Number of rules loaded: 69251.

Step 15. Disabling the systemd service.

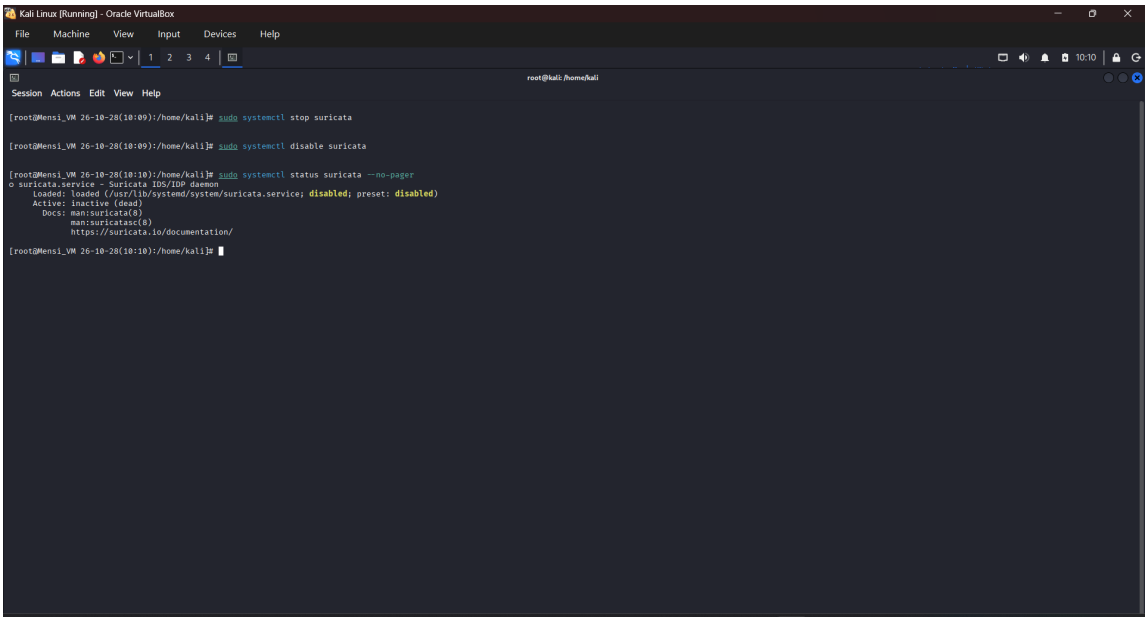


Figure 10: Suricata service stopped and disabled

Step 16–17.

```
# suricata -c /etc/suricata/suricata.yaml -T -v
# suricata -c /etc/suricata/suricata.yaml -v -i eth0
```

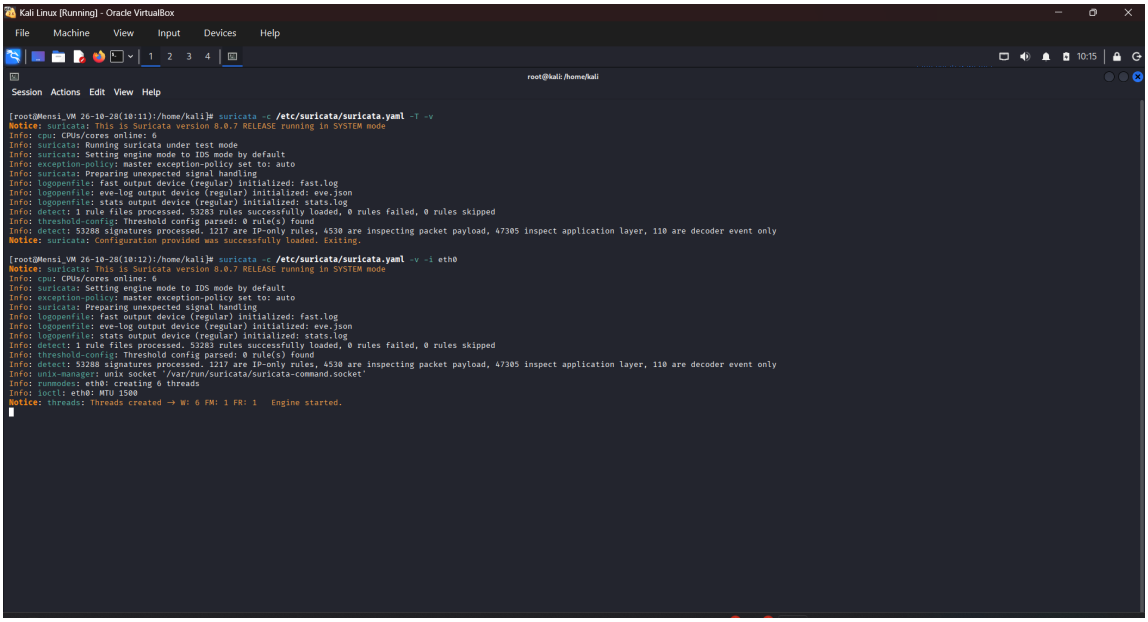


Figure 11: Successful configuration test and start (`-T -v`)

IV. Testing Suricata's Built-In Rules

IV.1 Detection of Root-Level Compromise

Step 18. Presence of rule SID 2100498.

```
# grep "sid:2100498" /var/lib/suricata/rules/suricata.rules
```

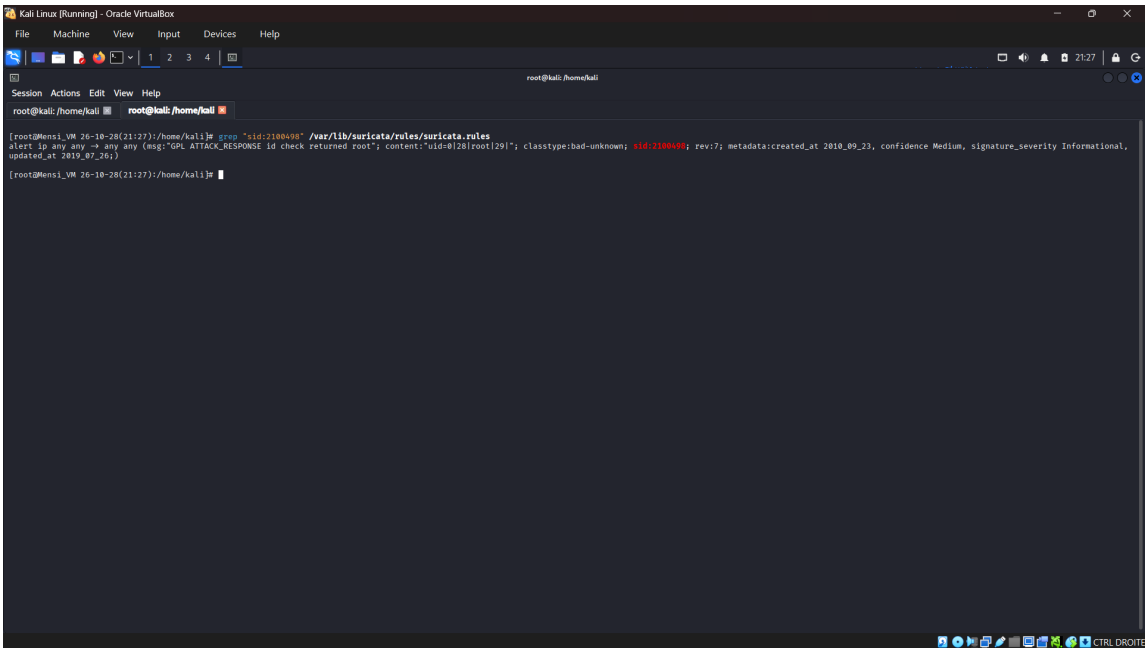


Figure 12: Rule SID 2100498 found in `suricata.rules`

This rule (SID 2100498) detects the output of the Unix `id` command when it returns root privileges, a common post-exploitation indicator showing that an attacker has command execution as root on the server. The header `alert ip any any -> any any` applies it to any protocol, address and direction. The `content` keyword matches the string `uid=0(root)`: `uid=0` and `root` are plain ASCII, while the parentheses are written in hexadecimal (`|28|` for "(" and `|29|` for ")").

Step 19. Simulation and alert.

```
$ curl testmynids.org/uid/index.html
# tail -f /var/log/suricata/fast.log
```

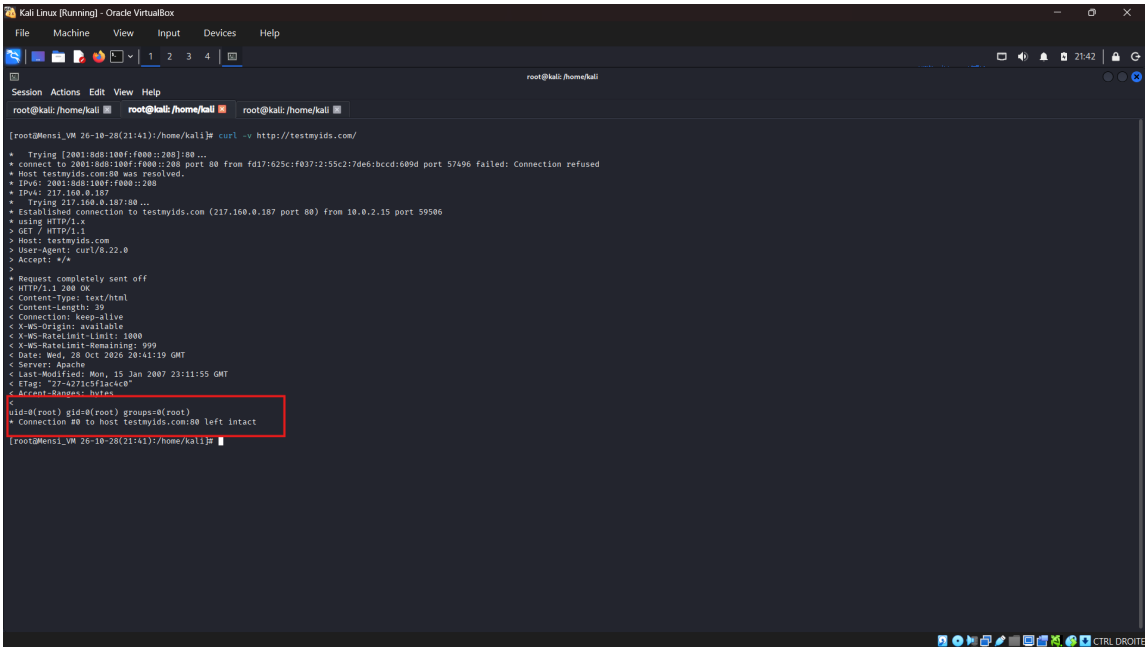


Figure 13: `curl` to the NIDS test page

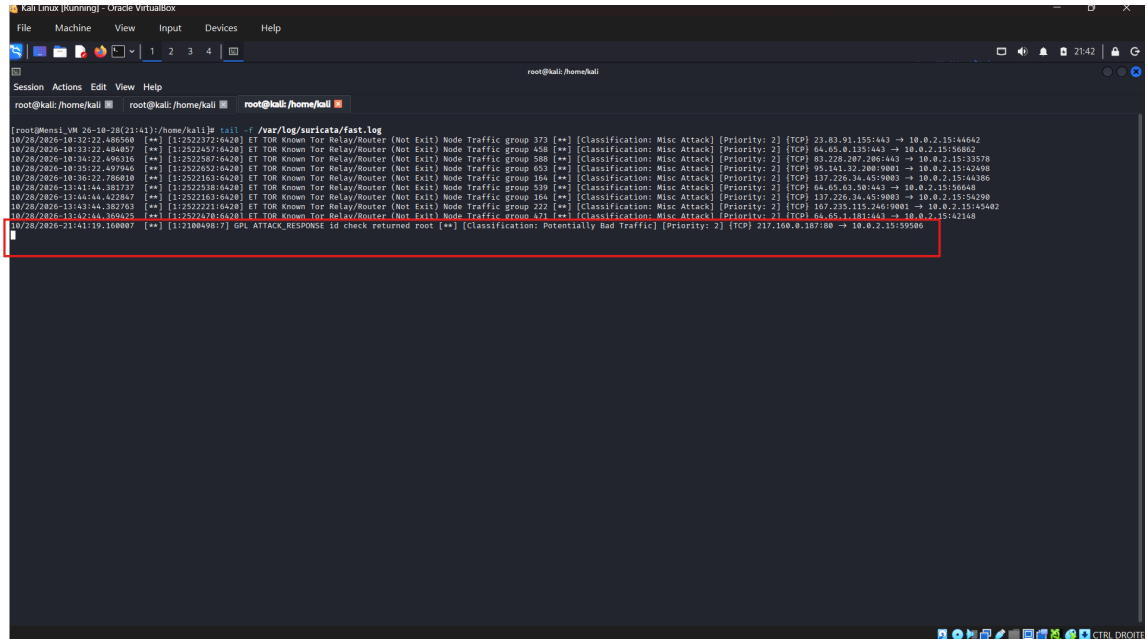


Figure 14: Alert generated for SID 2100498 in `fast.log`

Interpretation of the alert. The alert was generated by rule SID 2100498 when Suricata saw the string `uid=0(root)` in a TCP packet. The classification is *Potentially Bad Traffic* (`classtype:bad-unknown`) with priority **2**. The source is the public web server `217.160.0.187` on port 80, which belongs to `EXTERNAL_NET`, and the destination is my Kali VM `10.0.2.15` which belongs to `HOME_NET`. The traffic is therefore the HTTP *response* (server → client, inbound) to my `curl` request, whose body contained the output of the `id` command. In a real incident, this would indicate that a command executed with root privileges on a compromised server. Here it is a safe simulation using the benign test page `testmyids.com`.

IV.2 Detection of Malware User-Agents (BlackSun)

Step 20. Rule SID 2008983 – explanation of the key modifiers.

Keyword	Explanation
`flow:to_server,established`	Applies the rule only to packets of an established TCP session (handshake completed) going from the client to the server, so only the client's HTTP request is inspected.
`http.user_agent`	Sticky buffer: restricts the following `content` match to the value of the HTTP `User-Agent` header, which Suricata extracts with its HTTP parser.
`content:"BlackSun"`	The string searched for in that buffer: `BlackSun`, the User-Agent used by the malware to identify itself to its C2 server.
`nocase`	Makes the match case-insensitive, so `blacksun` or `BLACKSUN` also trigger the rule.
`depth:8`	Limits the search to the first 8 bytes of the buffer, the length of `BlackSun`, so the string must appear at the very start of the header. The rule shown in the lab uses `startswith`, which has the same effect.
`reference`	Adds a link to external information about the threat (here a Bitdefender page). It is shown to the analyst and has no effect on matching.
`classtype`	Assigns the alert to a category (`trojan-activity`). `classification.config` gives it the description "A Network Trojan was detected" and its priority.
`sid`	Unique signature ID (2008983). The 2000000 to 2999999 range is used by the Emerging Threats rules.
`rev`	Revision number of the rule (here 10), increased each time the rule is modified.

Step 21–22. HTTP request with the custom User-Agent.

```
$ curl -A "BlackSun" http://httpforever.com
```

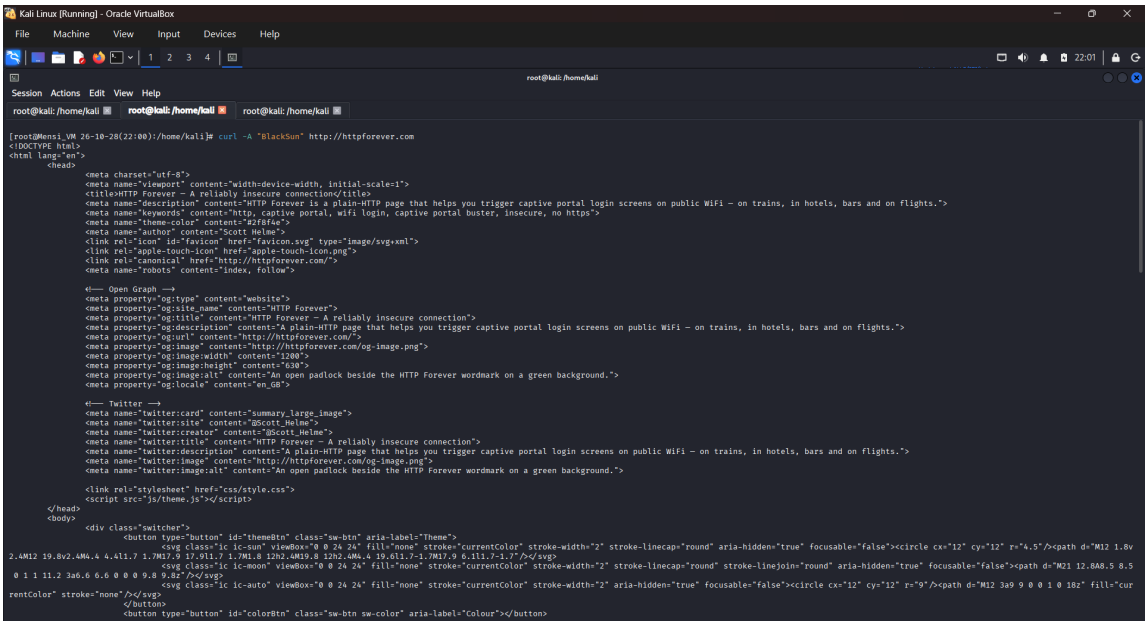


Figure 15: `curl` with User-Agent `BlackSun`

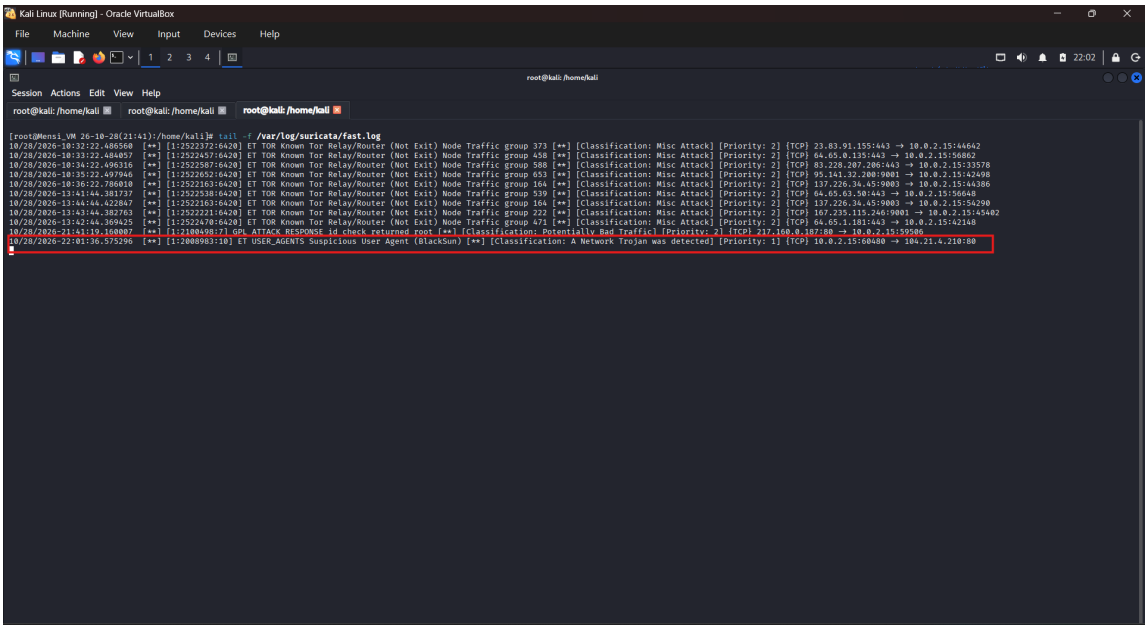


Figure 16: Alert for SID 2008983

Step 23. Two variants of the User-Agent.

```
$ curl -A "BlackSunBLABLABLA" http://httpforever.com
$ curl -A "BLABLABLABlackSun" http://httpforever.com
```

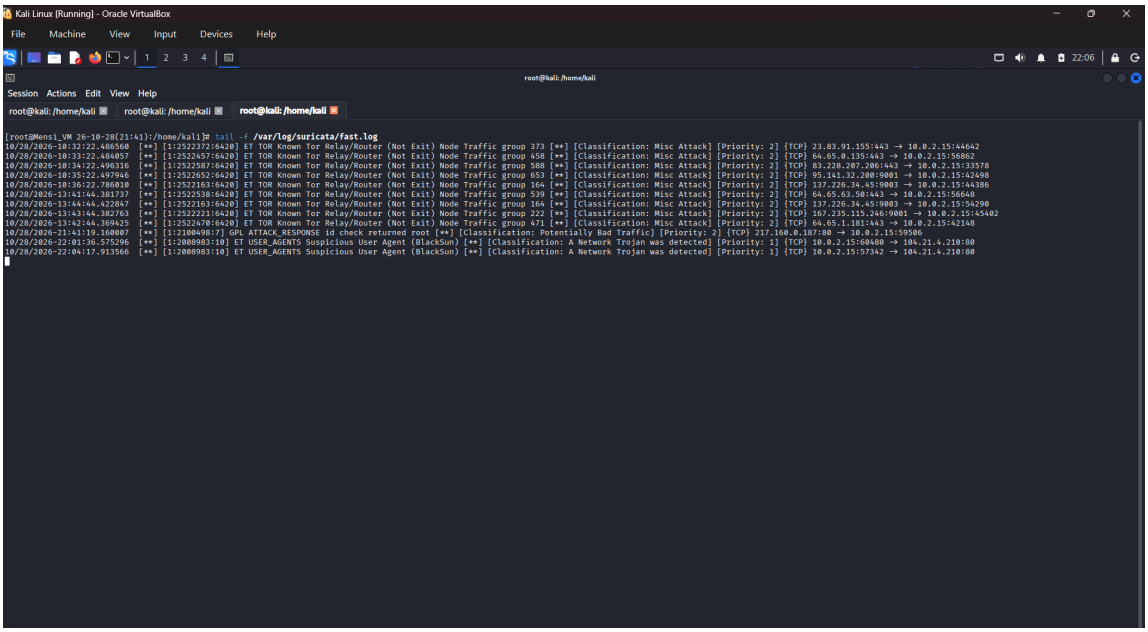


Figure 17: `fast.log` after both commands

Interpretation: as the screenshot shows, although two `curl` commands were executed, only one alert was generated, by the first command. The rule matches `BlackSun` only at the *beginning* of the HTTP `User-Agent` header, which is enforced by `startswith` (equivalent to `depth:8` with no offset, since `BlackSun` is 8 bytes long). The first User-Agent, `BlackSunBLABLABLA`, begins with `BlackSun`, so it matches. In the second, `BLABLABLABlackSun`, the string `BlackSun` appears at the end, not at the start of the header, so the positional constraint fails and no alert is raised even though the string is present in the header. This shows that a signature anchored to a position is precise but easy to evade: an attacker only needs to prepend a few characters to the User-Agent (if the malware and its C2 server accept it) to bypass this rule.

IV.3 Detection of Malicious IP Sources (Spamhaus DROP)

Step 24. Presence of the Spamhaus DROP rule (SID 2400000 and following).

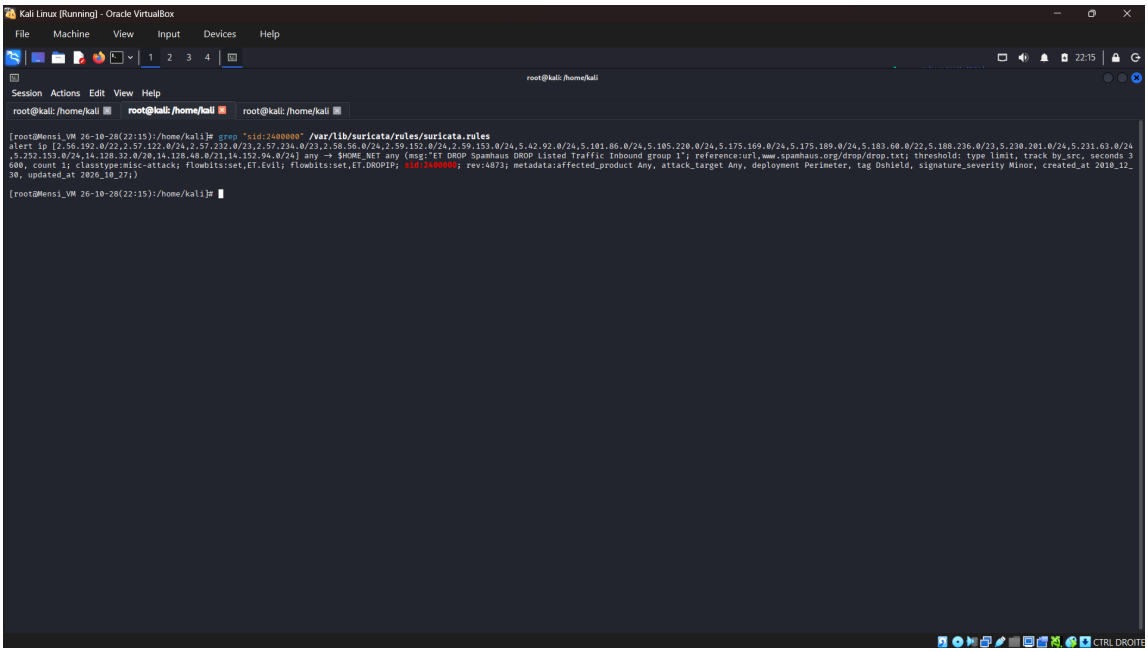


Figure 18: Spamhaus DROP rule in `suricata.rules`

Step 25. Explanation of the `threshold` keyword.

Keyword	Explanation
`threshold: type limit, track by_src, seconds 3600, count 1`	Limits the number of alerts generated by this rule: with `type limit`, `count 1` and `seconds 3600`, at most **1 alert** is logged per hour for each tracked source, even if the rule keeps matching. `track by_src` means the counter is kept per source IP address, so each distinct source address can raise its own alert. This prevents alert flooding during a DoS/DDoS attack (where the same rule would otherwise match thousands of packets), while still reporting every listed source at least once. The first match triggers the alert; subsequent matches from the same source are suppressed until the 3600-second window expires.

Step 26–27. SYN flood with spoofed random sources (closed port 2020, unused target IP).

```
$ sudo apt install hping3
$ sudo hping3 -S -p 2020 --rand-source -i u1000 10.10.10.10
```

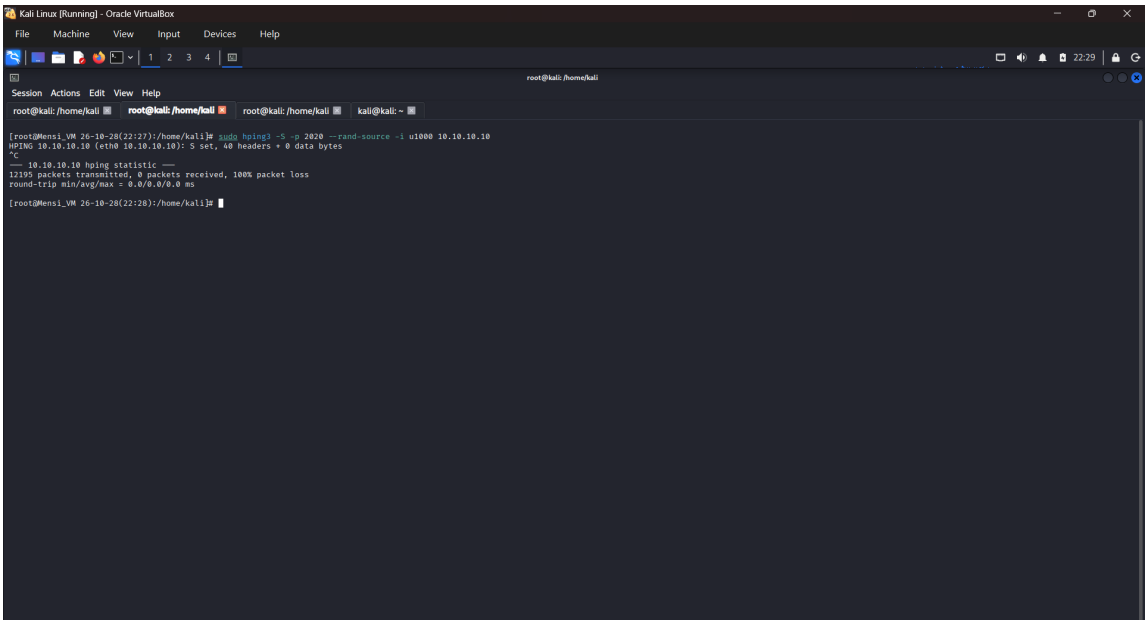


Figure 19: SYN flood launched with `hping3`

Step 28. Alerts observed.

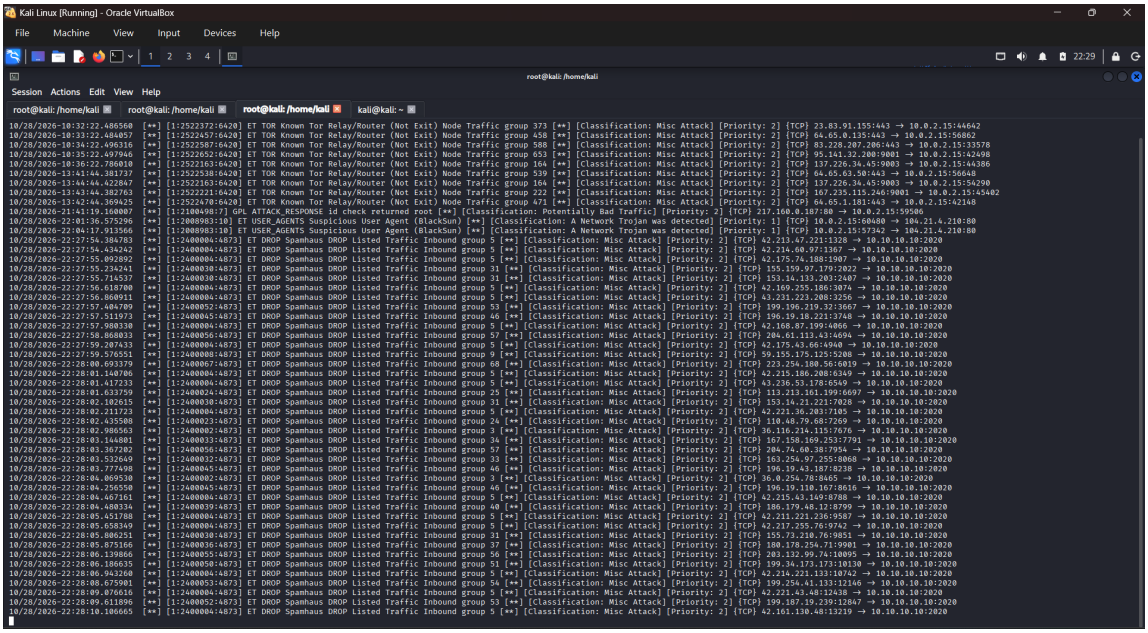


Figure 20: Spamhaus DROP alerts in `fast.log`

Why only some spoofed addresses trigger alerts. The `-rand-source` option of `hping3` assigns a different source IP to every packet, drawn at random from the entire IPv4 address space (roughly 4×10^9 addresses). The Spamhaus DROP rule only matches packets whose source address falls within one of the specific netblocks listed in the rule, which together cover a small fraction of the IPv4 space. Most spoofed packets therefore originate from addresses outside these ranges and simply do not match the signature they are not blocked or flagged, they just fall outside its scope. Only the subset of randomly generated sources that happen to land inside a listed Spamhaus netblock produces an alert.

Effect of the threshold on alert volume. Without the `threshold: type limit, track by_src, seconds 3600, count 1` clause, Suricata would log one alert per matching packet. Dur-

ing a flood, a listed source could be seen many times, which would quickly saturate `fast.log` with near-duplicate entries. The threshold keeps a separate counter per source IP (`track by_src`) and allows at most one alert per source every 3600 seconds (`count 1`). Every malicious source is therefore still reported at least once, but the alert volume stays manageable even if that source triggers the rule thousands of times within the hour.

V. Adding Our Own Rules to Suricata

Step 29–30. Creation and loading of `custom.rules`.

```
# touch /var/lib/suricata/rules/custom.rules
```

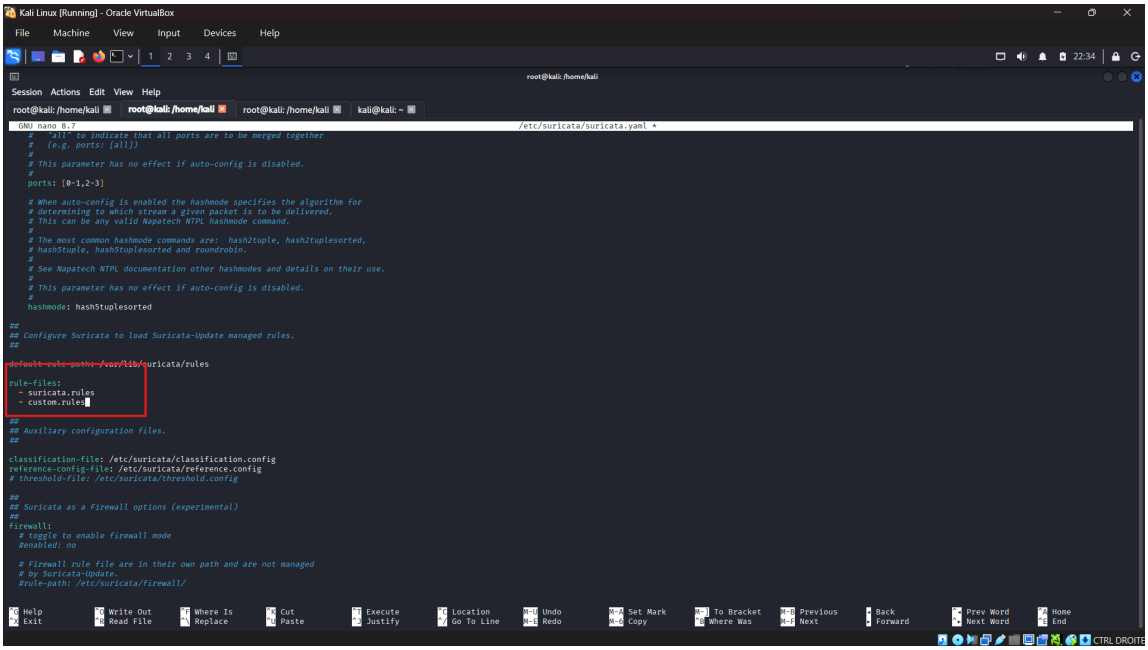


Figure 21: `custom.rules` added under `rule-files`

V.1 Detecting Echo Replies

Step 31. Rule appended to `custom.rules`:

```
alert icmp $EXTERNAL_NET any -> $HOME_NET any (itype:0; icode:0; msg:"Unauthorized Incoming ICMP
     Reply Detected"; classtype:attempted-recon; sid:1000001; rev:2;)
```

Step 32. Key modifiers.

Keyword	Explanation
`itype:0`	ICMP message type 0, which corresponds to an **Echo Reply**. This restricts the rule to the reply packets sent back after a ping, excluding the original Echo Request (type 8) and any other ICMP message type.
`icode:0`	ICMP code 0, the only valid code defined for an Echo Reply message. ICMP codes refine the meaning of a given type; since type 0 has a single associated code, this field is technically redundant here, but it is included for precision and consistency with how ICMP signatures are typically written.

Step 33–35. Restart and test.

```
$ ping -c 4 8.8.8.8
```

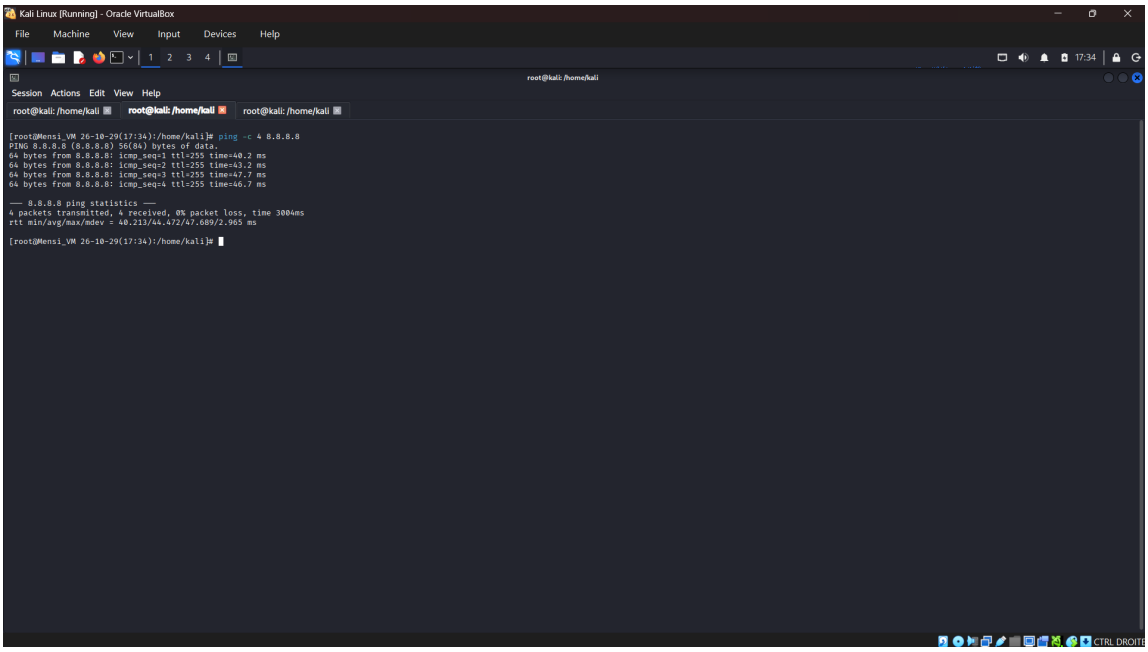


Figure 22: Ping to 8.8.8.8

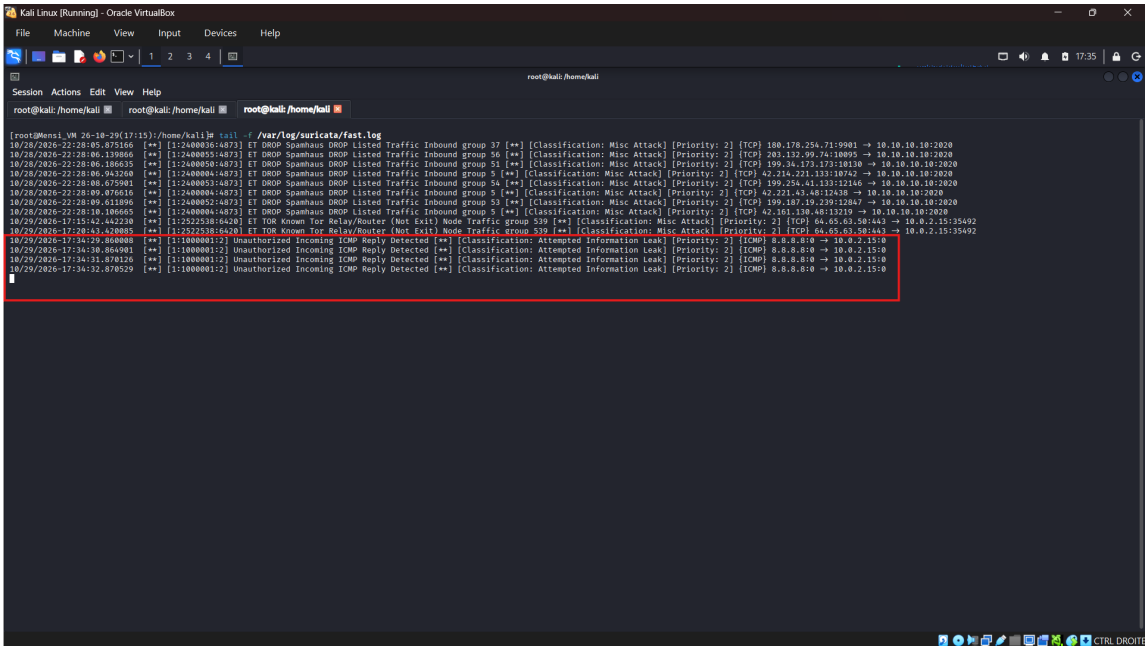


Figure 23: Alert for SID 1000001

V.2 Detecting ICMP Flood / Denial of Service

Step 36. Rule written (SID 1000002):

```
alert icmp $HOME_NET any -> $EXTERNAL_NET any (itype:8; msg:"Possible ICMP Flood / Ping Sweep
    Detected - Excessive Outgoing Echo Requests"; threshold:type threshold, track by_src, count
    20, seconds 1; classtype:attempted-dos; sid:1000002; rev:1;)
```

Justification. The direction `$HOME_NET any -> $EXTERNAL_NET any` matches outgoing traffic as required. `itype:8` selects ICMP Echo Request, the message type used in a ping flood. The `threshold` keyword uses `type threshold` (rather than `type limit` as in the Spamhaus rule): it only fires once the configured count is reached within the window, then resets, which matches the requirement of alerting *when* more than 20 requests occur within 1 second, rather than merely capping how many alerts get logged. `track by_src` counts requests per source IP (i.e., per host originating the flood); a limitation is that this does not restrict the count to a single destination, as `threshold` can only track by source or by destination, not by the source/destination pair – this is discussed further in step 38.

Step 37. Test.

```
$ ping -i 0.01 -c 50 8.8.8.8
```

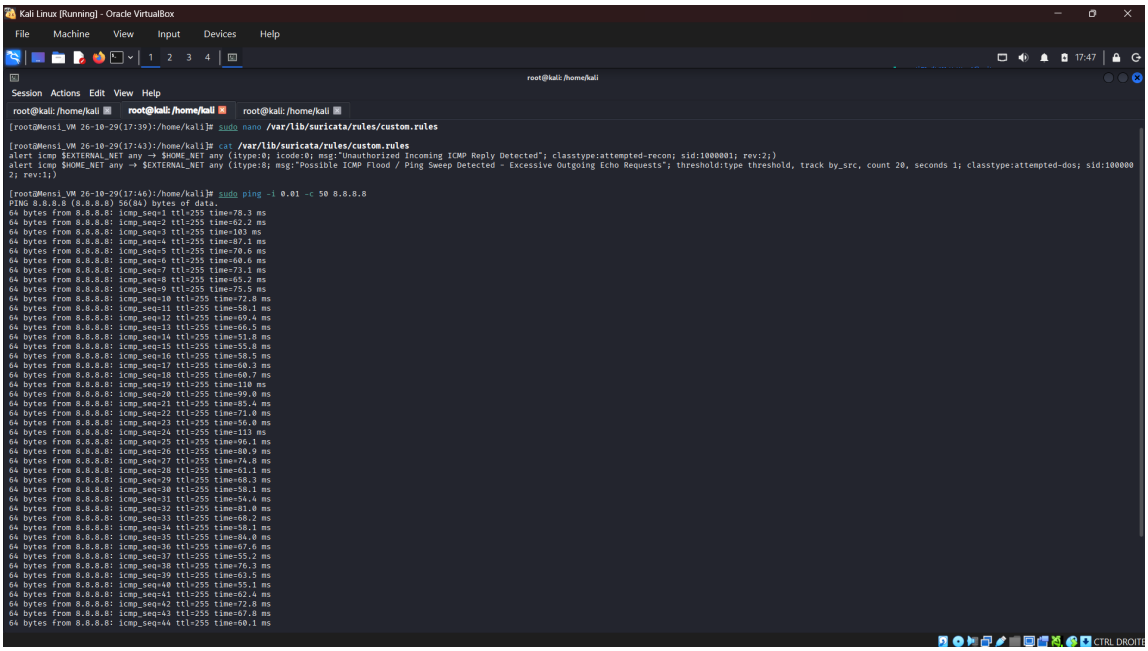


Figure 24: ICMP flood generation

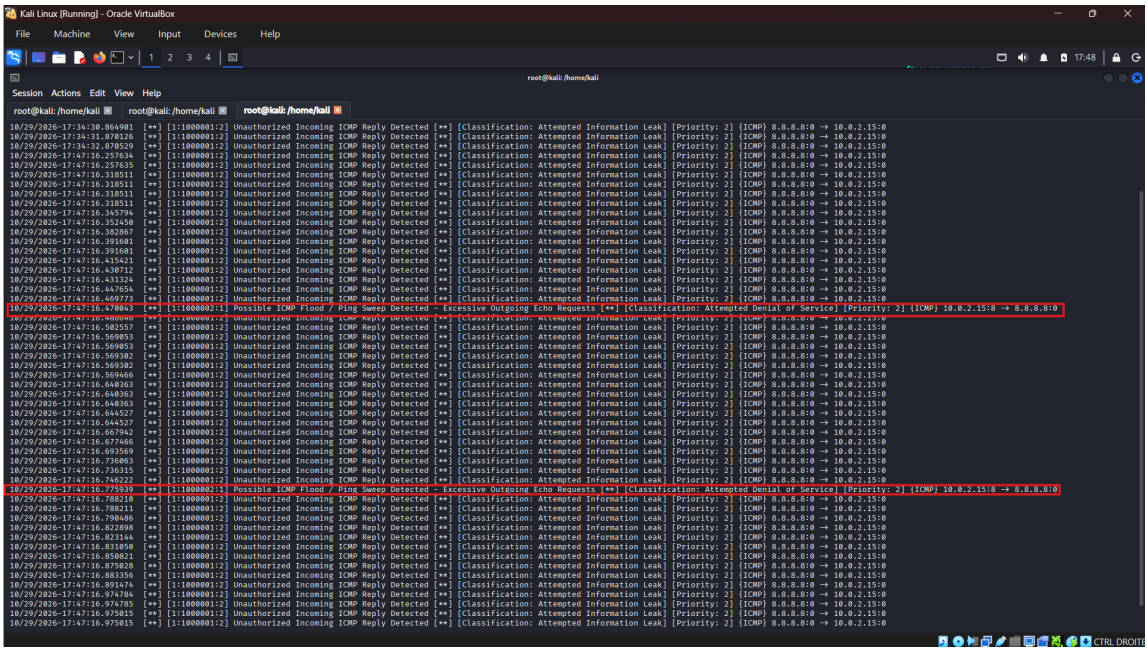


Figure 25: Alert for SID 1000002

Step 38. Risks of relying only on threshold-based detection. **Step 38.** Risks of relying only on threshold-based detection. Relying solely on a static count/time threshold to detect DoS activity has several weaknesses:

- **False negatives via low-and-slow evasion.** An attacker aware of the threshold (20 requests/second here) can simply stay below it – e.g. sending 19 pings per second indefinitely. The aggregate traffic can still degrade a target over time, yet no single window ever crosses the configured count, so the rule never fires.
- **False negatives via distributed sources.** Because `track by_src` counts per individual source IP, an attacker can spread the same total volume across many hosts (a botnet or spoofed sources), so that no single source ever reaches 20 requests/second even though the destination is collectively flooded. This mirrors the limitation already seen with the Spamhaus rule's `track by_src` threshold.
- **Evasion by changing destination / protocol.** The threshold here is scoped to ICMP Echo Requests only. An attacker can switch to UDP, TCP SYN, or application-layer floods (HTTP GET floods, DNS query floods) to achieve the same denial-of-service effect while bypassing this specific signature entirely, since it has no visibility into other protocols.
- **False positives.** Legitimate high-frequency ICMP traffic (e.g. network monitoring tools, `mtr`, automated health checks, or a user intentionally running diagnostic pings) can trigger the same rule, since the signature cannot distinguish benign bursts from malicious ones – it only measures rate, not intent.
- **Threshold tuning trade-off.** Setting the threshold too low increases false positives; setting it too high lets smaller but still harmful floods through. The "correct" value is environment-specific and not something a signature can determine on its own.

Observation. In practice, the rule fired twice rather than once during the 50-packet test. This is consistent with the semantics of `type threshold`: once the configured count is reached within the 1-second window, Suricata emits a single alert and resets both the counter and the timer. Because the 50 pings were not delivered in a single tight sub-second burst (due to scheduling and

virtualization overhead on the lab VM), the twenty-request threshold was independently crossed in two separate 1-second windows, producing two alerts instead of one.

These limitations illustrate why signature- and threshold-based detection is typically complemented with behavioral or statistical anomaly detection (e.g. NetFlow-based baselining), rate-limiting at the firewall, and correlation across multiple sources in a SIEM, rather than relying on a single Suricata rule.

V.3 Detecting PDF File Transfer

Step 39. PDF magic number.`xxd somepdf.pdf | head -n 1` (or `hexedit`, or Wireshark's packet bytes view).

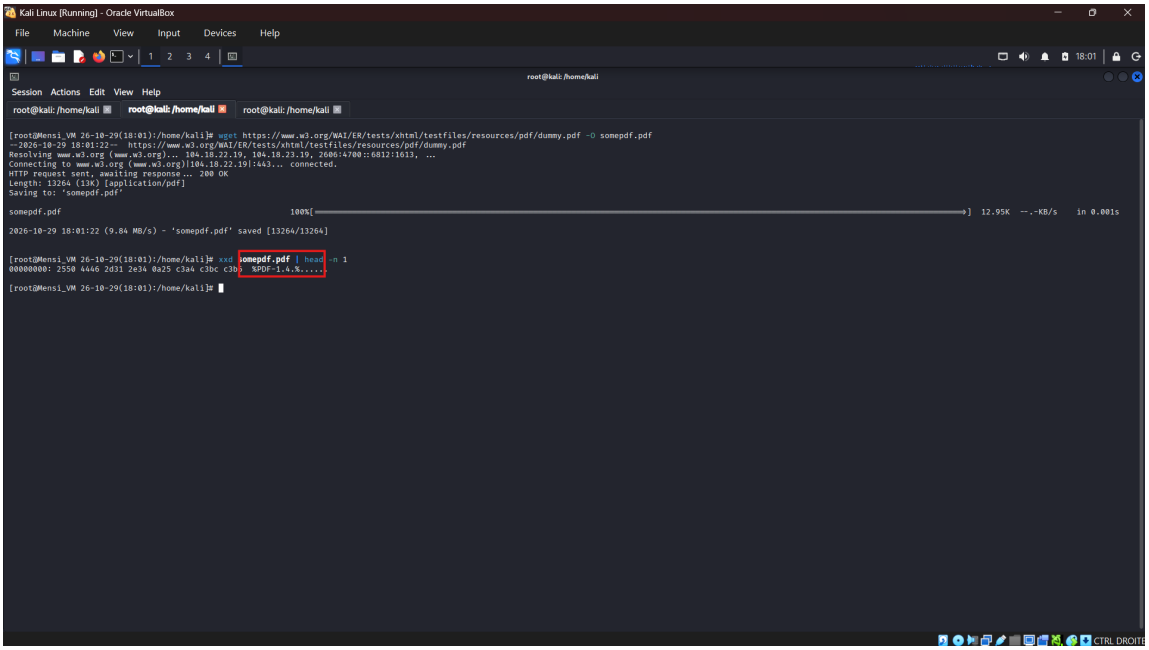


Figure 26: PDF opened in `bless`: signature `25 50 44 46 2D`

Step 40. Rule appended:

```
alert tcp any any -> $HOME_NET any (msg:"PDF file transfer detected"; content:"|25 50 44 46 2D
    |"; classtype:policy-violation; sid:1000003; rev:1;)
```

Step 41–42. Analysis of `with_pdf.pcap` in Wireshark.

- Display filter used: `tcp contains %PDF-`
- Packet numbers containing `%PDF-`: 17 and 42

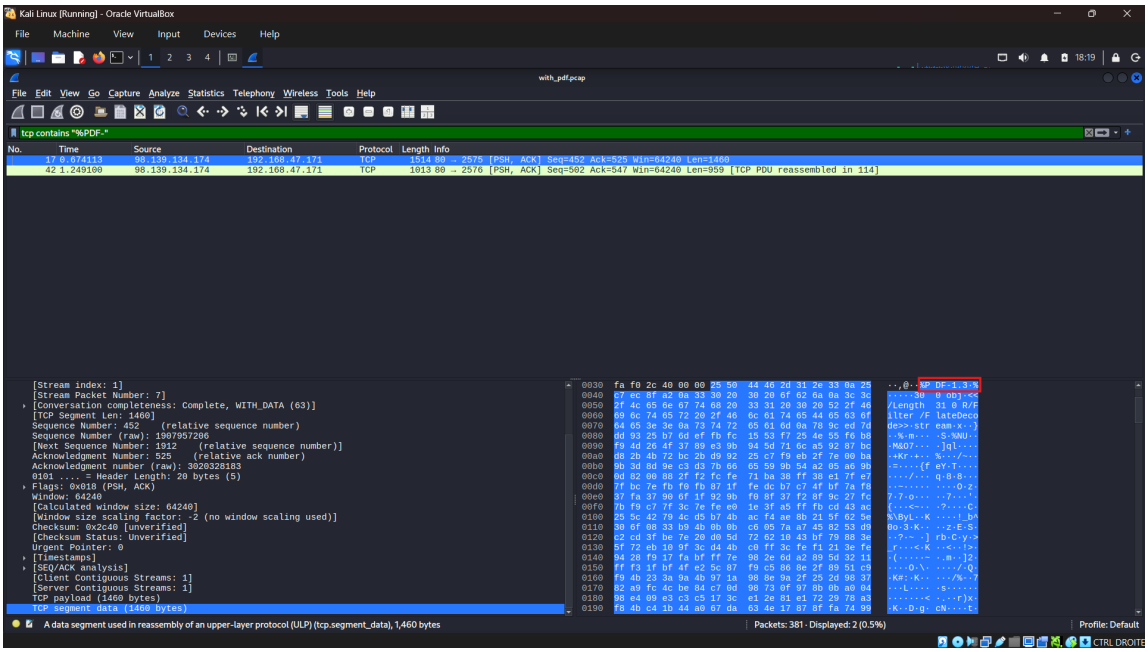


Figure 27: Packets containing the `%PDF-` header

Step 43–45. Replay and alert.

```
$ sudo tcpreplay -i enp0s3 with_pdf.pcap
```

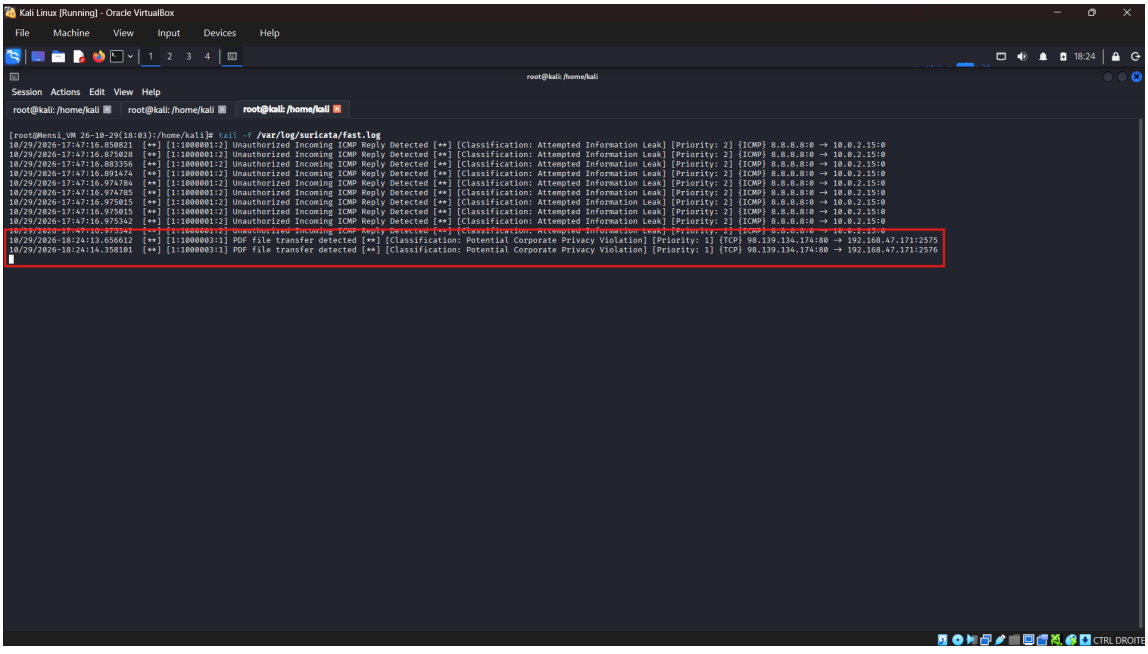


Figure 28: Alert for SID 1000003

Step 46. Usefulness and limitations of magic-number detection. **Step 46.** Usefulness and limitations of magic-number detection.

Why detect PDF transfers. PDFs are one of the most common vectors for phishing and malware delivery: attackers routinely attach malicious PDFs to phishing emails or host them for download, exploiting either social engineering (tricking the user into opening an attachment that looks legitimate) or technical vulnerabilities in PDF readers (embedded JavaScript, exploits

targeting the parser, malicious `/OpenAction` triggers). Detecting PDF transfers at the network level gives visibility into this delivery channel and acts as an early-warning signal, flagging file movements that can then be investigated or correlated with other indicators, even before any malicious payload inside the PDF is identified.

Limitations of magic-number (signature) detection.

- **Fragmentation/segmentation.** If the `%PDF-` header is split across two TCP segments (e.g. due to a small MTU or how the sender buffers the write), a rule using `content` without stream reassembly or `file.data` support may miss it, since it only inspects individual packets rather than the reassembled byte stream.
- **Header offset or obfuscation.** The PDF specification tolerates some leading bytes before `%PDF-` (e.g. a BOM or junk bytes), and a file can be deliberately padded or have its header shifted to avoid matching a rule that expects the signature at a fixed offset.
- **Encrypted or tunneled traffic.** Over HTTPS, the PDF bytes are encrypted at the TLS layer, so a plaintext-signature rule like this one cannot inspect the payload at all unless the sensor performs TLS interception/decryption.
- **No insight into malicious content.** The signature only confirms that *a* PDF was transferred – it says nothing about whether that PDF contains an exploit, embedded JavaScript, or is entirely benign. A malicious and a harmless PDF produce an identical alert.
- **False positives.** Because PDFs are extremely common in normal business traffic (invoices, reports, forms), this rule will alert on every PDF transfer, benign or not, generating significant noise without further context or a lower-priority classification to match.

Possible evasion techniques. An attacker can bypass this signature by: compressing or archiving the PDF (e.g. inside a ZIP or encoding it as Base64) so the raw `%PDF-` bytes never appear on the wire; modifying or corrupting the header bytes in a way the target PDF reader still tolerates but the signature does not match; encrypting the transfer (HTTPS, a VPN, or any TLS-wrapped channel); or deliberately fragmenting the file across many small packets/segments so the signature straddles packet boundaries and is never present intact within the inspection window of a single packet.

V.4 Detecting Embedded JavaScript in PDF Files

Step 47. Rule appended (SID 1000007):

```
alert http any any -> $HOME_NET any (file.data; content:"|25 50 44 46 2D|"; depth:5; content:"/
    JavaScript"; nocase; distance:0; within:8192; flow:to_client,established; msg:"Suspicious
    PDF: embedded JavaScript detected"; classtype:policy-violation; sid:1000007; rev:2;)
```

Step 48. Explain the Key parts of the Rule.

Keyword	Explanation
`file.data`	Sticky buffer that switches inspection from raw packet payload to the reassembled file content (the actual bytes of the file being transferred, e.g. the PDF body reconstructed by Suricata's HTTP/file-extraction engine), independent of how it was split across TCP segments or HTTP chunks. All `content` matches that follow apply to this buffer rather than individual packets.
`content:"|25 50 44 46 2D|"`	The first pattern searched for inside `file.data`: the hexadecimal bytes `25 50 44 46 2D`, i.e. the ASCII string `%PDF-`. This confirms the transferred file is a PDF, as in the earlier PDF-detection rule.
`depth:5`	Restricts the previous `content` match (`%PDF-`) to the first 5 bytes of `file.data`, i.e. it must appear at the very start of the file, which is where the PDF signature is defined to be.
`content:"/JavaScript"`	The second pattern searched for: the literal string `/JavaScript`, the PDF keyword used to embed JavaScript code or reference a JavaScript action inside the document.
`nocase`	Makes the `/JavaScript` match case-insensitive.
`distance:0`	Specifies that the search for `/JavaScript` starts immediately after the end of the previous match (the `%PDF-` header), with zero bytes skipped – i.e. it does not need to appear right after the header, but the search window begins there.
`within:8192`	Limits the search for `/JavaScript` to within 8192 bytes following the position set by `distance`, bounding how deep into the file Suricata looks for the keyword (for performance reasons, rather than scanning the entire file).
`flow:to_client,established`	Restricts the rule to the server-to-client direction (`to_client`) of an established TCP session, since the PDF file is being downloaded *from* the server (e.g. the Windows host serving the file) *to* the client (the Suricata sensor), matching the direction of the file transfer in this test.

Step 49–53. Transfer of the benign JS PDF (`JSPopupCalendar.pdf`) from the Windows machine.

```
C:\> python -m http.server 8000 # Windows
$ wget http://<windows_ip>:8000/JSPopupCalendar.pdf # Suricata host
```

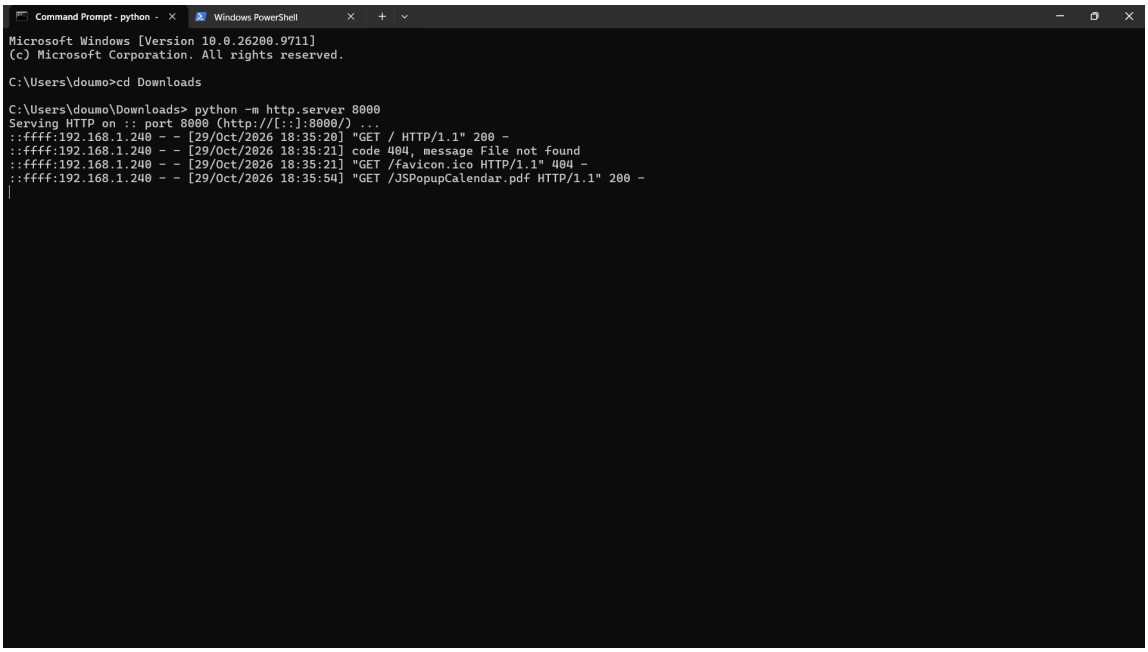


Figure 29: HTTP server running on the Windows machine

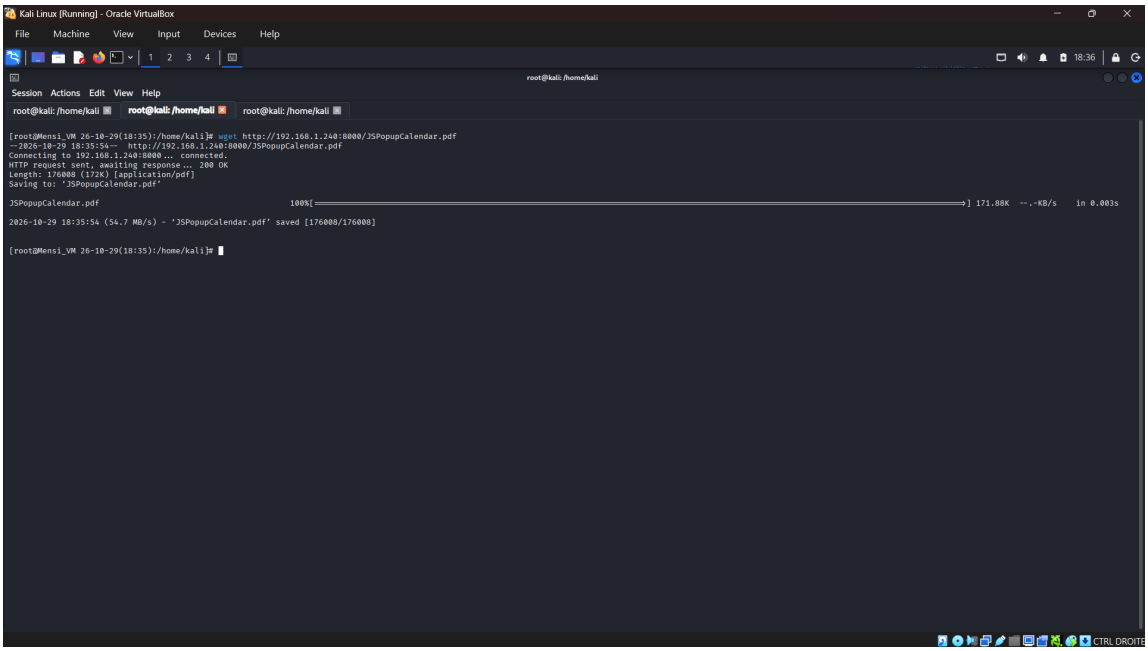


Figure 30: Download on the Suricata host

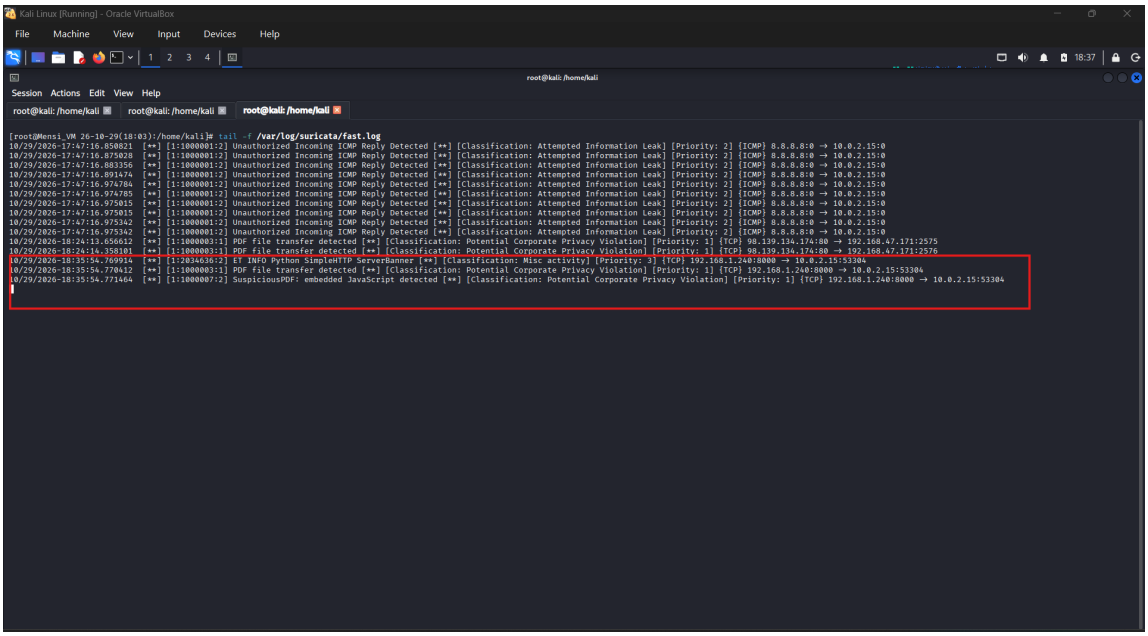


Figure 31: Alert for SID 1000007

V.5 Detecting EXE File Transfer

Step 54. How can an EXE be detected in network traffic? **Step 54.** Explain how you can detect the existence of an EXE file inside network traffic.

Windows executables follow the **Portable Executable (PE)** format, which contains several recognizable signatures that can be used for network-level detection:

- **MZ header.** Every PE file begins at offset 0 with the two bytes `4D 5A` (ASCII `MZ`), the signature of the legacy DOS executable header (named after Mark Zbikowski, one of the format's original designers at Microsoft). This is the simplest and most commonly used indicator for detecting EXE transfers, analogous to the `%PDF-` magic number used for PDFs.
- **DOS stub string.** Immediately following the MZ header, most PE files contain a small DOS-mode program whose purpose is to print `"This program cannot be run in DOS mode"` if the file is executed under plain DOS. This ASCII string is present in the vast majority of real-world EXE files and can serve as a secondary, more specific indicator to reduce false positives from the 2-byte `MZ` signature alone.
- **PE signature.** At a fixed offset stored within the MZ header (the `e_lfanew` field, typically found around offset 0x3C), the file contains the 4-byte signature `50 45 00 00` (ASCII `PE\0\0`), which marks the start of the actual PE/COFF header. This is the most precise indicator but requires parsing an offset rather than matching a fixed position, which is harder to express in a simple Suricata `content` match.

In practice, a Suricata signature combining the `MZ` header at a fixed offset (and optionally the DOS stub string shortly after it) over `file.data` gives a reasonable balance between detection reliability and rule simplicity, similar to the approach used for the PDF and JavaScript-PDF rules above.

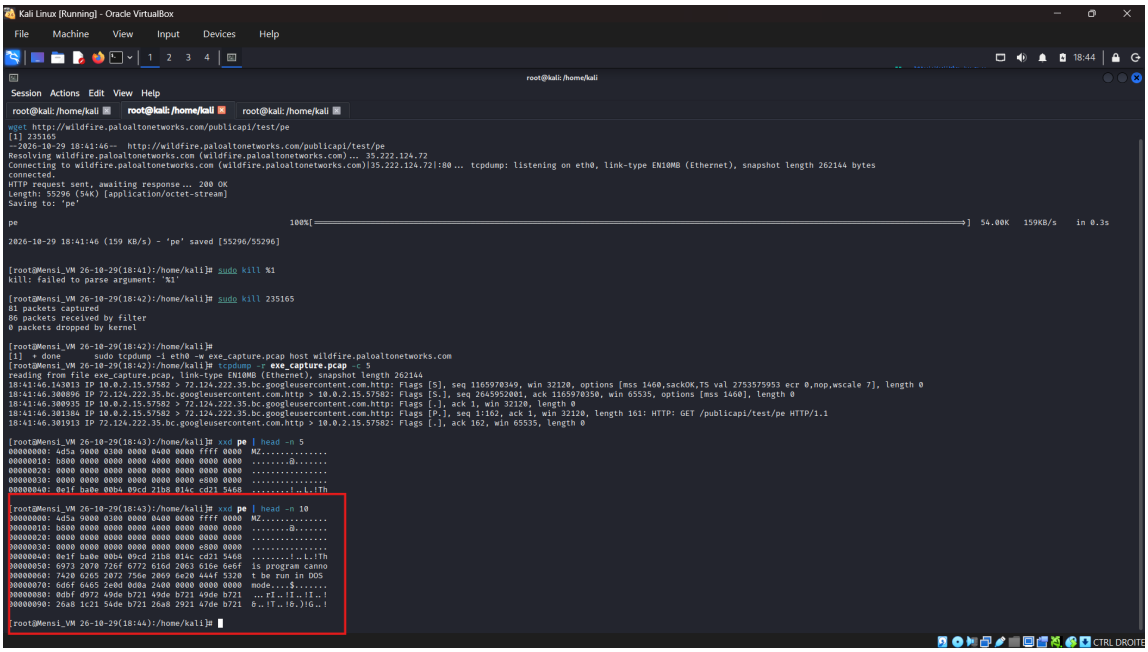


Figure 32: PE header of the WildFire sample (optional evidence)

Step 55. Proposed signature (SID 1000011):

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"Possible EXE File Transfer Detected (MZ
    Header)"; flow:to_client,established; file.data; content:"|4D 5A|"; depth:2; content:"This
    program cannot be run in DOS mode"; distance:0; within:200; classtype:policy-violation; sid
    :1000011; rev:1;)
```

Justification of each keyword.

- `$EXTERNAL_NET any -> $HOME_NET any` – matches the direction of a download: an external server sending a file to a host on the internal network, the same direction used by the PDF/JavaScript rules.
- `flow:to_client,established` – restricts inspection to the server-to-client side of an established TCP session, i.e. the HTTP response carrying the file, not the client's request.
- `file.data` – switches inspection to the reassembled file content extracted by Suricata, rather than raw packet payload, so the signature still matches even if the file is split across multiple TCP segments.
- `content:"|4D 5A|"` with `depth:2` – matches the `MZ` magic bytes required at the very start of any PE file, bounding the match to the first 2 bytes of `file.data` so it only matches a genuine header position, not an incidental occurrence of those bytes later in the file.
- `content:"This program cannot be run in DOS mode"` with `distance:0, within:200` – a second, more specific indicator: the DOS stub message present in the vast majority of real PE files, searched for within the 200 bytes following the `MZ` header (where the DOS stub conventionally sits). Requiring both patterns together substantially reduces false positives compared to matching `MZ` alone.
- `classtype:policy-violation` – consistent with how the lab classifies unauthorized/restricted file-type transfers (PDF, JavaScript-PDF rules above).
- `sid:1000011`, `rev:1` – unique signature ID in the custom range used throughout this report, initial revision.

Discussion of false positives. The 2-byte `MZ` signature alone is a weak indicator: those two bytes can appear by chance in many binary formats, especially if matched without `depth` or outside `file.data`. Requiring both `MZ` at offset 0 *and* the DOS stub string shortly after substantially narrows the match to genuine PE files, since the stub is specific to the Windows executable format. However, false positives remain possible: any file type that embeds a legitimate PE (e.g. installers, drivers, or files repackaging an EXE, such as self-extracting archives or certain container formats) would also match, even though the top-level file transferred may not itself be flagged as "an EXE download" in the way an analyst expects. Additionally, this rule says nothing about whether the executable is malicious – benign software downloads (legitimate installers, updates) will also trigger it, so in a real deployment this would typically be tuned with allow-lists for known-good distribution servers or combined with reputation/sandboxing for the extracted file.

Step 56. Test.

```
$ wget http://wildfire.paloaltonetworks.com/publicapi/test/pe
```

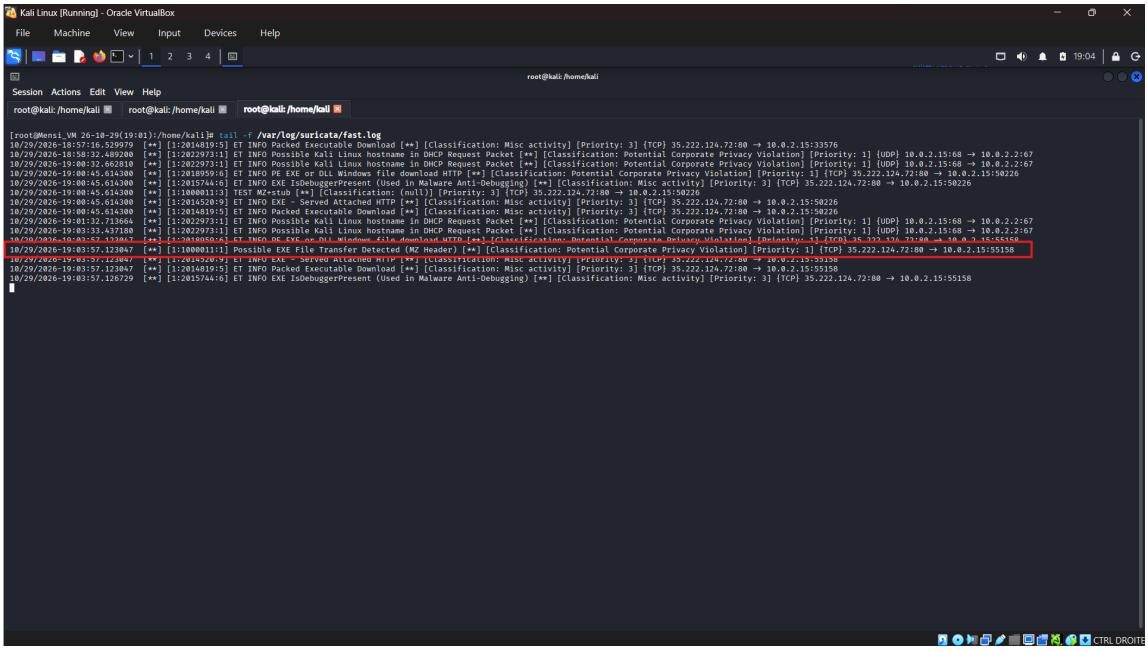


Figure 33: Alert generated by the EXE rule

V.6 Detecting Clear-Text Card Numbers (DLP)

Step 57. Rule appended (SID 1000009):

```
alert tcp $HOME_NET any -> any 25 (flow:to_server,established; smtp.body; pcre:"/4\d{3}(\s|-)?\d
    {4}(\s|-)?\d{4}(\s|-)?\d{4}/"; msg:"Sensitive Data Exposure: Possible Visa Card Number
    Detected in Cleartext"; classtype:policy-violation; sid:1000009; rev:1;)
```

Step 58. Explain the Key parts of the Rule.

Keyword	Explanation
`smtp.body`	Sticky buffer that restricts the following match (here, the `pcre`) to the body of an SMTP message, i.e. the actual email content, as parsed by Suricata's SMTP protocol parser – excluding headers, MIME boundaries, and envelope data.
`pcre:"/4\d{3}(\s|-)?\d{4}(\s|-)?\d{4}(\s|-)?\d{4}/"`	Perl-Compatible Regular Expression matching a 16-digit sequence that starts with `4` (the Visa prefix), grouped into four blocks of four digits, with an optional single space or hyphen allowed between each group: `4\d{3}` matches the first digit (4) followed by exactly 3 more digits; `(\s|-)?` optionally matches one space or hyphen as a separator; this block of 4 digits plus optional separator then repeats three more times, covering formats such as `4xxx-xxxx-xxxx-xxxx`, `4xxx xxxx xxxx xxxx`, and `4xxxxxxxxxxxxxxx` with no separators at all.

Step 59–60. Wireshark analysis of `email_cc2.pcap`.

- Does the capture contain SMTP traffic? yes
- Reconstruction of the email via *Follow TCP Stream*.
- Evidence of a Visa number in plaintext (packet 15).

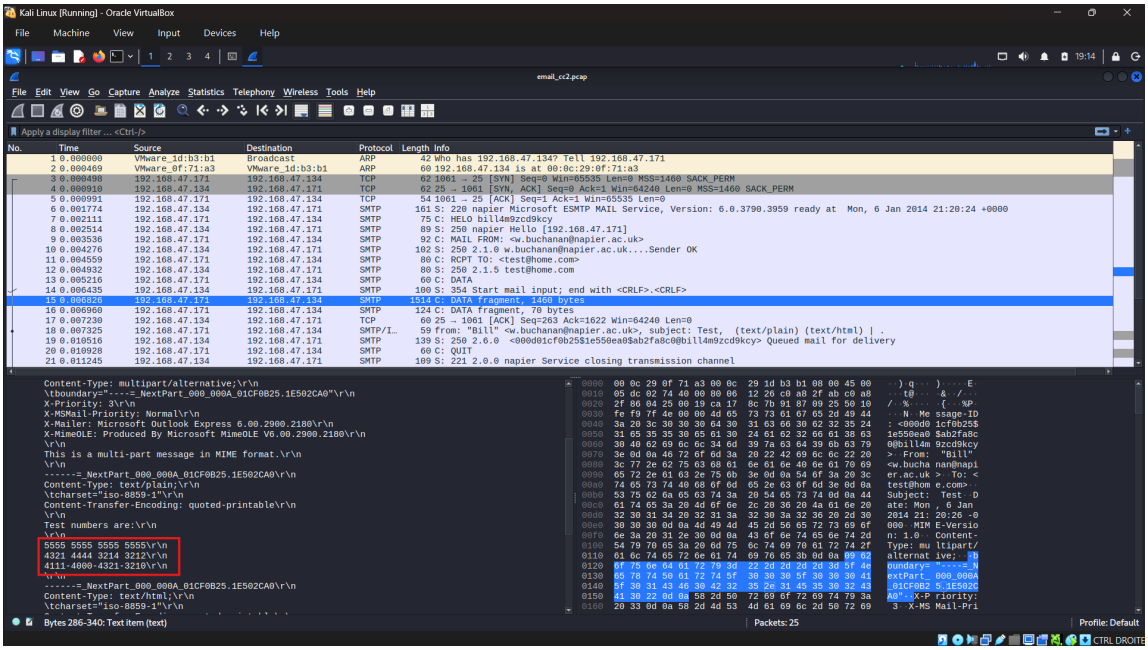


Figure 34: SMTP packet 15 showing the card numbers

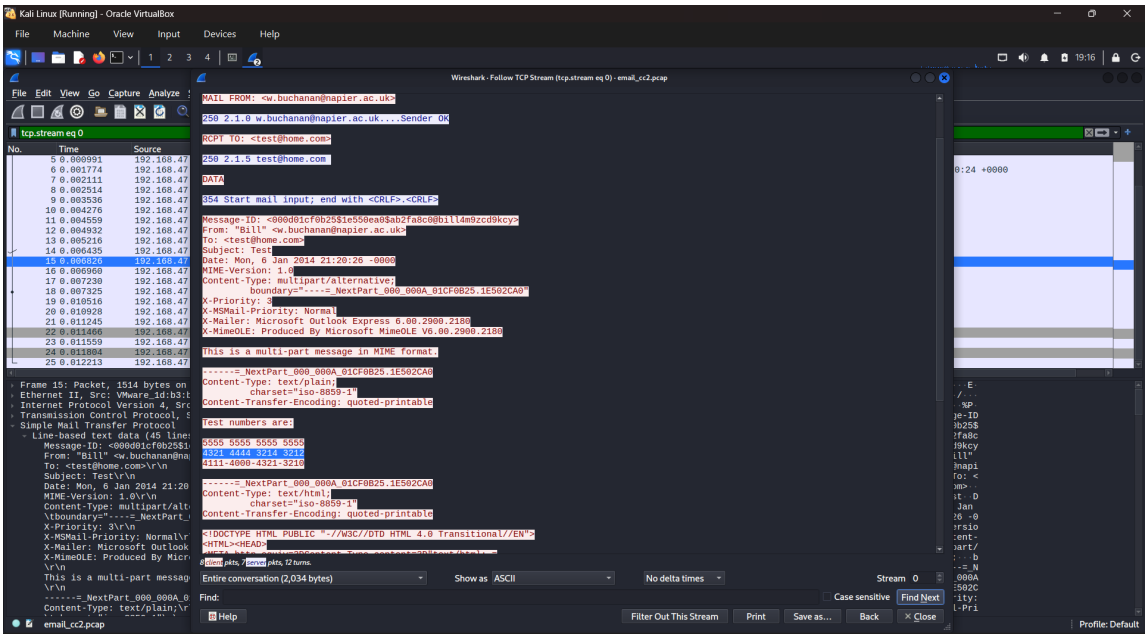


Figure 35: Reconstructed email (Follow TCP Stream)

Step 61–63. Replay and alert.

```
$ sudo tcpreplay -i enp0s3 email_cc2.pcap
```

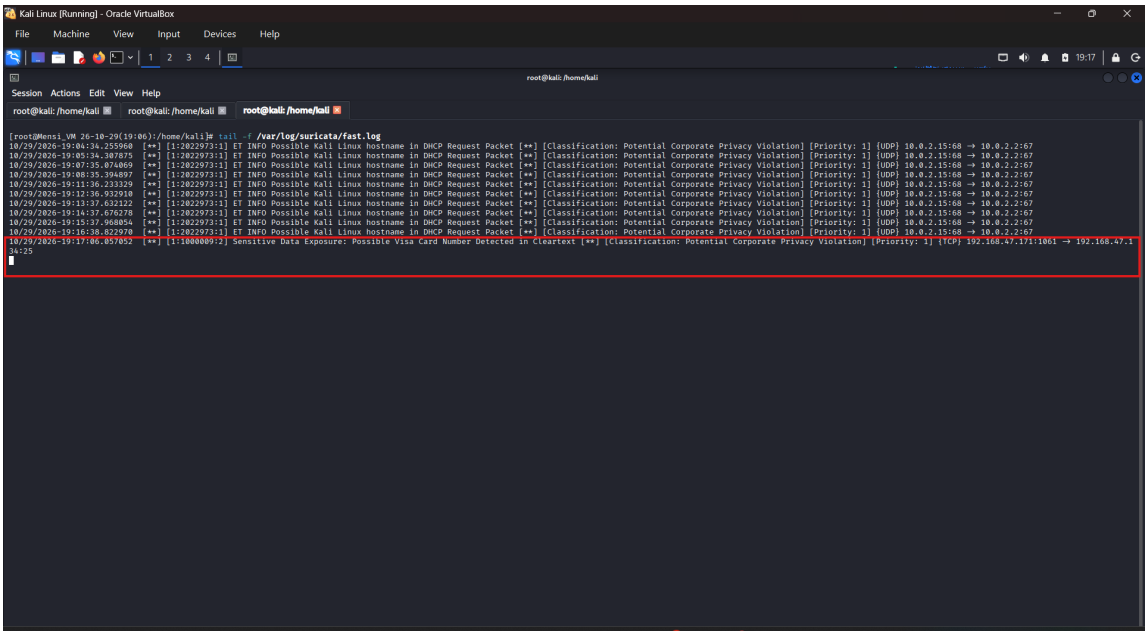


Figure 36: Alert for SID 1000009

Matching against the regex. The email body contains three test numbers:

- `5555 5555 5555 5555` – does *not* match: the regex requires the first digit to be `4` (the Visa prefix), but this number starts with `5` (the Mastercard range), so the `4\d{3}` anchor fails immediately.
- `4321 4444 3214 3212` – *matches*: starts with `4`, is 16 digits total, grouped in blocks of 4 separated by single spaces, exactly as the pattern `4\d{3}(\s|-)?\d{4}(\s|-)?\d{4}(\s|-)?\d{4}` expects.

- `4111-4000-4321-3210` – *matches*: starts with `4`, 16 digits, grouped in blocks of 4 separated by hyphens, which the `(\s|-)?` alternation also accepts.

Of the three numbers present in the email, two trigger the rule and one does not – confirming that the regex is specifically anchored to the Visa prefix (`4`) and does not generalize to other card brands.

Step 64. Limitations of regex-based detection and DLP evasion.

Limitations.

- **False positives.** The regex matches *any* 16-digit sequence starting with 4 and grouped in the expected pattern – it performs no Luhn checksum validation, so a random number, an order ID, a phone-adjacent sequence, or a fabricated test value with the right shape and prefix will trigger an alert even though it is not a real, valid card number.
- **Narrow coverage.** The rule only covers the Visa prefix (`4`). Mastercard (prefix 5x), American Express (34/37), Discover (6011), and other card brands are entirely invisible to this signature, as demonstrated by the `5555...` number above passing through undetected.
- **Protocol/port scope.** The rule is scoped to TCP port 25 (SMTP) only. Card numbers sent via webmail (HTTPS), file-sharing links, chat applications, or any other channel are outside this rule's visibility entirely.
- **Encryption.** Modern mail transport commonly uses STARTTLS to upgrade a plaintext SMTP session to TLS. Once encrypted, the payload is opaque to a signature-based IDS without TLS interception, rendering this detection useless against properly configured mail servers.
- **Performance cost.** PCRE evaluation is significantly more expensive than simple `content` byte matching, especially on high-throughput links; if applied broadly (e.g. to all TCP payload rather than a narrow buffer) it can become a bottleneck under load.

Evasion techniques. An attacker (or data exfiltration malware) could bypass this rule by:

- Encoding or obfuscating the number (Base64, ROT13, splitting digits with unusual separators the regex doesn't anticipate, inserting zero-width or invisible characters between digits).
- Splitting the card number across multiple separate SMTP messages or multiple TCP segments/sessions so no single payload ever contains the full 16-digit sequence intact.
- Embedding the number inside an attachment (e.g. an image, PDF, or encrypted ZIP) rather than the plaintext message body, which this rule does not inspect.
- Using an encrypted channel (STARTTLS, a webmail HTTPS session, or a different protocol entirely) to remove visibility from a plaintext-focused signature.
- Formatting the number with separators or digit groupings the regex's alternation does not account for (e.g. dots, multiple spaces, or digits written as words), causing the pattern match to fail despite the underlying data still being sensitive.

These limitations illustrate why signature-based DLP on raw network traffic is typically only one layer of a broader data-protection strategy, complemented by endpoint DLP agents (which can inspect data before encryption or compression), egress proxy inspection with TLS interception, and content-aware policies at the application layer (e.g. at the mail gateway or CASB level) rather than relying solely on a network IDS signature.

V.7 Analyzing a PCAP File and Detecting Phishing

Step 65–66. Packet 29 of `phishingattack.pcap`.

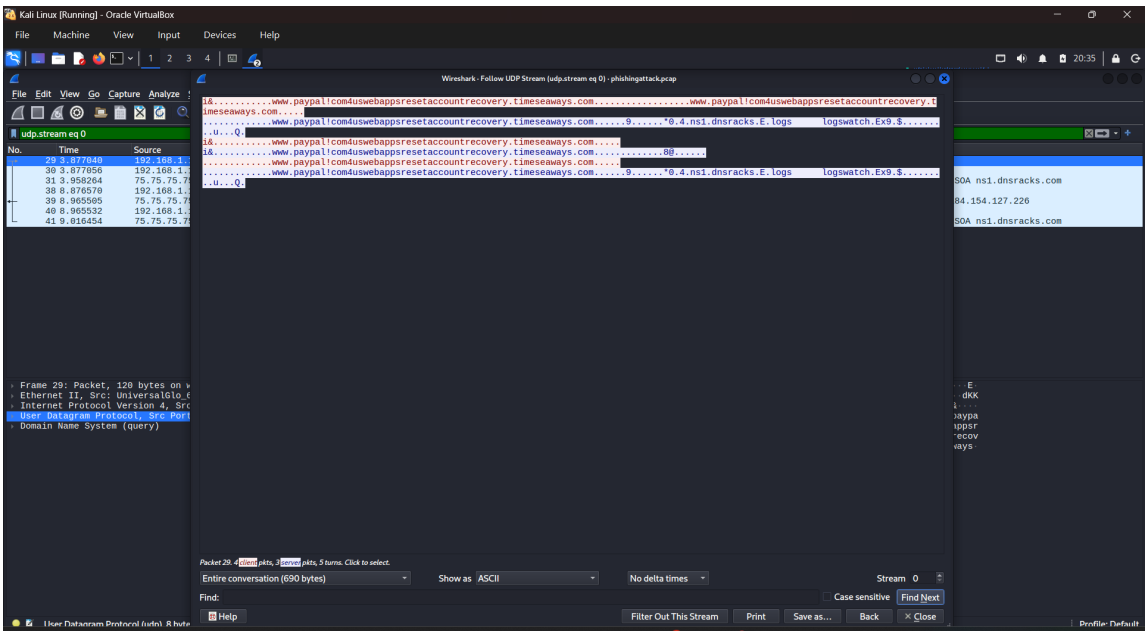


Figure 37: DNS query in packet 29

Does it look like phishing? Step 66. Does this look like a phishing attempt? Explain!

Yes, this is a textbook phishing/domain-spoofing attempt. The queried name is:

`www.paypal.com4uswebappsresetaccountrecovery.timeseaways.com`

To understand why this is deceptive, it is necessary to read a DNS name from *right to left*: the actual registered domain is always the rightmost two labels (the second-level domain plus the top-level domain), with everything to the left being a subdomain label chosen freely by whoever controls that domain. Here, the real registered domain is `timeseaways.com` – an attacker-controlled domain with no connection to PayPal. Everything before it, including the substring `www.paypal.com4uswebappsresetaccountrecovery`, is a single long subdomain label of `timeseaways.com`.

The attacker deliberately embeds the legitimate-looking string `paypal.com` near the start of this label (`www.paypal.com4uswebapps...`) so that, at a glance – especially on a small screen, in a rushed email client preview, or to a user who only skims the beginning of the URL – it visually resembles a genuine PayPal address (`www.paypal.com/...`). In reality, `paypal.com` here is not a domain at all; it is just part of a longer subdomain string, syntactically no different from any other word an attacker could insert. The rest of the label (`4uswebappsresetaccountrecovery`) reinforces the social-engineering pretext, mimicking the kind of path a legitimate "reset your account" flow might use, further increasing the illusion of legitimacy.

This is a well-known technique called **subdomain-based domain spoofing** (or "combosquatting" when combined with brand-name insertion). Because DNS has no concept of a "contained" or "related" domain – only exact ownership of the rightmost registered name – any attacker who owns `timeseaways.com` can create arbitrarily many subdomains containing any string they like, including the names of real companies, without any technical barrier. A user who trusts the presence of `paypal.com` in the string, rather than checking which domain the name actually resolves under, can easily be misled into visiting the attacker's infrastructure instead of PayPal's.

Step 67–68. Rule appended (SID 1000010):

```
alert dns $HOME_NET any -> $EXTERNAL_NET 53 (msg:"Phishing Attempt: Suspicious PayPal Domain
    Detected"; dns_query; content:"paypal.com"; nocase; isdataat:1,relative; classtype:trojan-
    activity; priority:2; sid:1000010; rev:1;)
```

Step 69. Key parts of the rule.

Keyword	Explanation
`dns_query`	Selects the DNS query buffer for inspection.
`isdataat:1,relative`	Checks whether at least one byte exists after the previous content match.

Step 70–72. Replay and alert.

```
$ sudo tcpreplay -i enp0s3 phishingattack.pcap
```

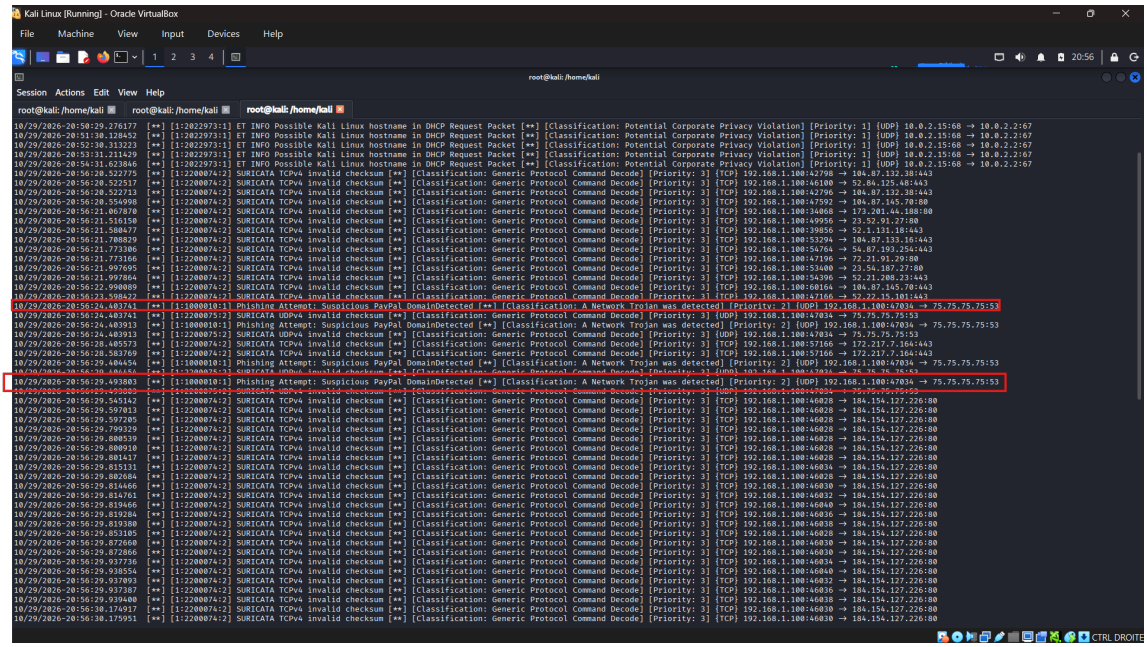


Figure 38: Alert for SID 1000010

Limitation: The rule would not normally flag the legitimate `www.paypal.com`, because the match for `paypal.com` occurs at the end of the DNS query, leaving no additional byte for `isdataat:1,relative` to detect. However, a suspicious domain such as `www.paypal.com4uswebapps.com` could match because additional characters follow the matched string. This condition reduces some false positives but does not prove that a domain is malicious. Conversely, a typosquatting domain such as `paypa1.com` would not be detected because it does not contain the exact string `paypal.com`. Detecting such lookalikes requires additional rules or techniques, such as edit-distance matching, domain reputation checks, or threat-intelligence feeds.

VI. Summary of Custom Rules and Alerts

SID	Purpose	Tested with	Alert?
1000001	Incoming ICMP echo reply	`ping 8.8.8.8`	yes
1000002	ICMP flood (outgoing)	ping -i 0.01	yes
1000003	PDF transfer	`with_pdf.pcap`	yes
1000007	PDF with JavaScript	`JSPopupCalendar.pdf`	yes
1000011	EXE transfer	WildFire sample	yes
1000009	Visa number over SMTP	`email_cc2.pcap`	yes
1000010	Phishing PayPal domain	`phishingattack.pcap`	yes

Table 3: Custom rules written/tested in this lab

VII. Conclusion

This lab provided practical experience with Suricata, from installation and network configuration to testing built-in signatures and developing custom detection rules. By analyzing network traffic and replaying PCAP files, we implemented rules to detect ICMP activity, suspicious file transfers, embedded JavaScript in PDFs, potential exposure of payment card numbers, and phishing-related DNS queries. The results demonstrated the effectiveness of signature-based detection in identifying known patterns and generating actionable alerts. However, these rules remain limited by false positives, false negatives, encryption, and evasion techniques that modify or conceal recognizable patterns. Their effectiveness also depends on appropriate network configuration, threshold tuning, and careful rule design. To improve detection capabilities, Suricata could be complemented with anomaly-based detection, file extraction and analysis, Lua scripting, and SIEM correlation. Overall, this lab highlighted the importance of combining precise signatures with broader contextual analysis to achieve more reliable network intrusion detection.